XIMOD Architect — XML Builder

ユーザーマニュアル

あなたのMODのために設計されたXMLインストーラー。

Skyrim · Fallout · Starfield · Oblivion · Morrowind

Version 2.2.0

クロスプラットフォーム：Windows · Linux · macOS



目次

1. 概要 6
   1.1 XIMOD Architectとは 6
   1.2 FOMOD形式 6
   1.3 インストールと動作要件 7
      Windows 7
      Linux 7
      macOS 8
   1.4 ファイル構成 8
2. はじめに 9
   2.1 アプリケーションの起動 9
   2.2 インターフェースの概要 9
   2.3 新規プロジェクトの作成 10
   2.4 プロジェクトを開く・保存する 10
   2.5 初回起動 11
   2.6 フリーウィンドウとキーボード操作 11
3. メニュー 12
   3.1 ファイルメニュー 12
   3.2 編集メニュー 13
   3.3 プロジェクトメニュー 14
   3.4 ツールメニュー 14
   3.5 ヘルプメニュー 15
4. 機能の詳細 17
   4.1 Mod情報タブ 17
   4.2 インストール手順タブ 18
      グループと選択タイプ 18
      オプションとデフォルトタイプ 19
      プラグインファイル 19
      グループ／ページの宛先 19
      条件フラグ 20
   4.3 必須のインストールタブ 20

4.4 条件付きインストールタブ 21
4.5「設定」ウィンドウ 21
4.6 保存前・保存後スクリプト 22
例 — 保存前スクリプト（Windows） 23
例 — 保存後スクリプト（Windows） 23
Linux / macOSでの同等の処理 23
4.7「バージョン情報」ウィンドウ 24
4.8 FOMODファイルの生成 24
4.9 翻訳エディター 24
4.10 XMLエディター 26
4.11 フォント管理 27
4.12 2つのFOMODの統合 28
4.13 配布用アーカイブのエクスポート 28
4.14「プロパティ」ウィンドウ 28
4.15 コマンドラインモード（CLI） 30
4.16 更新の確認 30
5. FOMODタグリファレンス 31
5.1 info.xmlファイル 31
5.2 ModuleConfig.xmlファイル 31
ルートとヘッダー 31
ファイルとフォルダ 32
ステップとグループ 32
オプション（plugin） 32
依存関係と条件 33
6. ModuleConfig.xmlファイルの詳細解説 34
6.1 インストーラーがファイルを読む仕組み 34
6.2 ルートとヘッダー 34
6.3 ファイルのインストール 35
6.4 ステップ、グループ、オプション 35
6.5 オプションのタイプとその挙動 36

6.6 フラグ、依存関係、条件付きインストール 37
7. ModuleConfig.xmlの例 39
7.1 最小の例 — 必須ファイル 39
7.2 排他的な選択 — SelectExactlyOne 39
7.3 条件付きインストール — フラグ 39
7.4 1ページ、複数ステップ — SelectAny + フラグ 40
7.5 カスタマイズ可能なデフォルト付きプリセット 42
7.6 導入ページと終了ページ 44
8. バージョン2 — 新機能 46
8.1 宛先衝突の検出 46
8.2 7zエクスポート 46
8.3 再利用可能なテンプレート 46
8.4 フォルダから新規 46
8.5 ドラッグ&ドロップでのファイル割り当て 46
8.6 FOMODの比較 47
8.7 ビジュアル条件エディター 47
8.8 画像のチェックと最適化 47
8.9 コマンドラインの追加 47
8.10 バックグラウンドでのエクスポートと検証 47
8.11 クラッシュからの復旧 47
8.12 より分かりやすいインターフェース 47
8.13 既存のFOMODの翻訳 48
8.14 MODアーカイブを開く 49
8.15 インポートの忠実性 49
8.16 バックアップと復旧 50
8.17 マスターを自動条件に 50
8.18 ライトプラグイン（ESL）とプラグインレポート 50
8.19 より豊かなシミュレーターとシナリオ 51
8.20 オプションごとのインストールサイズ 51
8.21 Nexus説明文ジェネレーター 52

8.22 編集可能な条件エディター 52
8.23 プロジェクトの文字列の表 52
8.24 BSAおよびBA2アーカイブの読み取り 53
8.25 入れ子の条件グループとバージョン条件 53
8.26 ゲームとそのカテゴリの編集 55
8.27 ツリーの色 55
8.28 ファイルダイアログが前回の場所を記憶 55
8.29 プリセットウィザード 56
8.30 FOMOD間での切り取り、コピー、貼り付け 56
8.31 ディスク上でのソースの名前変更 57
8.32 欠落ファイルの検索 57
付録 58
付録A — Config.iniファイル 58
付録B — キーボードショートカット 58
付録C — 用語集 60
付録D — 完全なFOMODの例 61
D.1 info.xml 61
D.2 ModuleConfig.xml 61
付録E — ライセンスとクレジット 65

1. 概要

1.1 XIMOD Architectとは

XIMOD Architectは、ベセスダのゲームMOD（Skyrim、Fallout、Starfield、Oblivion、Morrowindなど）向けのFOMODインストーラーを作成するためのクロスプラットフォームツールです。完全なインストールウィザード（ステップ、オプショングループ、オプション、条件付きファイル）を視覚的に構築し、MODマネージャーが解釈できる標準化されたXMLファイルを自動的に生成できます。

FOMODはMODマネージャーの形式であり、特定のゲームの形式ではありません。作成したインストーラーは、すべてのベセスダのゲームと、VortexまたはMod Organizer 2が管理できる他のほとんどのゲームで動作します。VortexでMODを導入できるゲームであれば、FOMODインストーラーを使用できます。Categories.jsonファイルの組み込みリスト（ベセスダの11タイトル：Skyrimの3つのエディション — Legendary、Special Edition、VR — 、Fallout 3、Fallout: New Vegas、Fallout 4、Fallout 4 VR、OblivionおよびOblivion Remastered、Morrowind、Starfield）は、info.xmlのゲームとNexusカテゴリを入力するためだけのものであり、インストーラーを使用できる場所を制限するものではありません。このリストは外部にあり編集可能です。ツール → ゲーム / カテゴリ…（§ 8.26）で、アプリケーションを再コンパイルすることなく他のゲームを追加できます。

本ソフトウェアは、オリジナルのFOMOD Creation ToolをRust言語で書き直したものです。この刷新により、パフォーマンスの向上、Windows・Linux・macOSでの同一の動作、そしてモダンなインターフェースが実現しました。XIMOD Architectは、非常に高度な国際化システムも特長としています。インターフェースは多数の言語に翻訳でき、各文字体系に適したフォントが動的に読み込まれます。

主な機能は次のとおりです。

- 完全なFOMODインストールパッケージの作成。
- 複数ステップのインストールウィザード。
- さまざまな選択タイプ（正確に1つ、少なくとも1つ、任意など）を持つオプショングループ。
- ユーザーの選択に基づく条件付きファイルインストール。
- オプションのタイプを決定する依存関係パターン。
- インストールの流れを制御する「フラグ」（条件フラグ）のシステム。
- 保存の前後に実行されるカスタムスクリプト。
- 多言語対応インターフェースと動的フォント。
- ネイティブな透明スプラッシュスクリーン。

1.2 FOMOD形式

FOMOD（「Fallout Mod」）は、ベセスダのゲーム向けMODインストーラーの標準形式であり、VortexやMod Organizer 2などのMODマネージャーは、他のほとんどのゲームでもこの形式を使用します。FOMODイン

ストーラーは、fomodという名前のサブフォルダにまとめられた2つのXMLファイルで構成されています。

- info.xml — MODのメタデータ：名前、作者、バージョン、ウェブサイト、説明、カテゴリ。
- ModuleConfig.xml — インストールの構成：ステップ、グループ、オプション、ファイル、条件。

XIMOD Architectが生成するファイルは、主要なMODマネージャーと互換性があります。

- Mod Organizer 2（MO2）。
- Vortex。
- Nexus Mod Manager（NMM）。

> 注 —
> fomodフォルダの名前、ならびにinfo.xmlおよびModuleConfig.xmlという名前は仕様で定められており、MODマネージャーはそのまま認識します。これらは決して変更してはいけません。

1.3 インストールと動作要件

システム要件（目安）：

項目	推奨
システム	Windows 10以降、Linux（最近のglibc）、またはmacOS 11以降。
ディスプレイ	OpenGL 3.3対応のグラフィックカード（eguiの「glow」レンダラー）。
RAM	数百MB — 本アプリケーションは軽量です。
ディスク容量	数十MB程度。Notoフォントとassets/に含まれる旗の数によって異なります。

Windows

実行ファイルximod-architect.exe、assetsフォルダ（data、locales、fonts、images、icons）、および必要に応じてスプラッシュスクリーン用のファイルassets/images/splash.pngを同じフォルダに配置します。アプリケーションのアイコンはコンパイル時に実行ファイルへ埋め込まれます。その後、ximod-architect.exeを実行してください。

Linux

インストールスクリプトがpackaging/linuxに用意されています。

```
# System-wide installation (requires sudo)
sudo ./packaging/linux/install.sh

# Installation for the current user
./packaging/linux/install.sh --user
```

ソースからのビルドにはRust 1.99.0以降が必要で、Linuxでは次の開発用ライブラリが必要です。libgtk-3-dev、libxcb-render0-dev、libxcb-shape0-dev、libxcb-xfixes0-dev、libspeechd-dev、libxkbcommon-dev、libssl-dev、libx11-dev。

macOS

配布可能なアプリケーションバンドル（.app）を作成するスクリプトが用意されています。

```
./packaging/macos/create-bundle.sh
```

「XIMOD Architect.app」バンドルがdistフォルダに生成されます。その後、hdiutilユーティリティを使ってディスクイメージ（.dmg）を作成できます。

1.4 ファイル構成

実行ファイルに付属するファイルツリーは次のとおりです。

```
ximod-architect(.exe)            → executable
Config.ini                       → configuration (created on first launch)
assets/
   data/
      Categories.json            → games and Nexus categories
      Languages.json             → 6,777 languages (endonym, ISO codes, font)
      Countries.json             → 244 countries (names, flag, official languages)
      CountryLanguages.json      → languages spoken by country
   locales/                      → translations (one folder per language)
      eng/main.ftl
      fra/main.ftl
      …                          → 32 languages provided
   fonts/                        → Noto fonts (one per writing system)
   icons/                        → application icons
   images/
      splash.png                 → splash screen
      svg/                       → country flags (244 SVG files)
```

2. はじめに

2.1 アプリケーションの起動

起動時、XIMOD Architectはまず、フェード効果付きのネイティブな透明スプラッシュスクリーンを表示します。この表示時間は調整可能です（デフォルトは2秒。スプラッシュスクリーンは無効にできます）。その後、プライマリディスプレイの中央にメインウィンドウが開きます。初期サイズは設定ファイルに基づき（デフォルトは1280×800）、800×600未満にはできません。タイトルバーには、アプリケーション名とそのバージョン番号が表示されます。

2.2 インターフェースの概要

メインウィンドウは5つの領域で構成されています。

- 上部のメニューバー（ファイル、編集、プロジェクト、ツール、ヘルプ）と、そのすぐ下にある日常的な操作用のツールバー（新規作成、開く、保存、元に戻す、やり直し、検証、プレビュー、エクスポート）。第3章を参照してください。
- 左側のプロジェクトツリー：MOD情報、続いてインストールステップとそのグループおよびオプション、必須ファイル、条件付きインストール。ノードをクリックすると編集でき、右クリックで操作（追加、複製、移動、削除、テンプレートとして保存）を行えます。オプションを別のグループへ、グループを別のステップへ、ステップを別のステップの前へドラッグして移動できます。「インストールステップ」と「条件付きインストール」の横にある+ボタンで要素を直接追加できます。
- 中央のインスペクター：選択したノードのエディターを表示します。MOD情報、ステップ（名前、表示条件、ページの宛先、グループの一覧）、グループ（名前、選択タイプ、グループの宛先、オプションの一覧）、オプション（説明、画像、タイプ、フラグ、依存関係パターン、ファイル）、必須ファイル、条件付きセット。インスペクター上部のパンくずリストで1つ上の階層に戻れます。
- 下部の「問題」パネル：プロジェクト→FOMODを検証の後に表示されます。各検出結果が深刻度とともに一覧表示され、クリックすると該当するノードがツリー内で選択されます。ツリー自体にも、該当するノードの横に小さなエラー／警告のカウンターが表示されます。
- 最下部のステータスバー：左側には最新のメッセージ（起動時は「準備完了」）、右側にはステップとオプションの数、そしてプロジェクトに未保存の変更がある場合はドットに続けて「変更あり」が表示されます。成功、警告、エラーは右上隅に通知としても表示されます。

インターフェースの外観（ダーク、ライト、またはシステムテーマ、フォントサイズ、言語）は、§ 4.5で説明する「設定」ウィンドウで設定します。

ツリーでは、クリックするとノードが選択されてインスペクターに表示され、ステップまたはグループをダブルクリックすると同様に選択されて展開されます。ステップの右クリックメニューには展開（すべて、選択項目、選択項目からステップとその配下すべて）と折りたたみ（同じ3つの選択肢）があり、グループのメニューには展開と折りたたみ（すべて、選択項目）があります。これらは通常の追加／複製／移動／削除コマンドの前に表示されます。各タブはそれぞれ独自の「問題」パネルを保持します。表

示される検出結果は常にアクティブなタブで読み込まれているFOMODのものであり、あるタブで開始した検証は、途中で別のタブに切り替えてもそのタブで完了します。

ツリーは、テキストフィールドにフォーカスがない間はキーボードでも操作できます。↑と↓はカーソルリングを行から行へ移動させ（選択中のノードから開始します）、Enterはその下の行を選択し、→は閉じているステップやグループを展開し（開いている場合は最初の子へ移動します）、←は開いているものを折りたたみます（閉じている行やオプションの場合は親へ戻ります）。リングは移動に合わせて画面内にスクロールされます。

2.3 新規プロジェクトの作成

インストーラーの作成は、一般的に次の手順に従います。

- 新規プロジェクト — ファイルメニュー → 新規作成を選び、MODのファイルが入ったルートディレクトリを選択します（Mod情報タブの「参照…」ボタン）。
- 情報の入力 — Mod情報タブで、MOD名、作者、バージョン、ゲーム、カテゴリ、ウェブサイト、ヘッダー画像、説明を入力します。
- ウィザードの構築 — インストール手順タブで、ステップ、オプショングループ、オプションの順に追加します。
- ファイルの関連付け — 各オプションについて、インストールするファイルまたはフォルダを追加し、必要に応じて表でその宛先を設定します（デフォルトは空で、ゲームのDataフォルダのルートにインストールされます）。
- 条件の定義 — フラグと条件付きインストールを使って、ユーザーの選択に応じてインストールを適応させます。
- 保存 — ファイルメニュー → 保存（Ctrl+S）：XIMOD Architectがfomodフォルダを作成し、その中にinfo.xmlとModuleConfig.xmlを書き込みます。

2.4 プロジェクトを開く・保存する

保存すると、常にルートディレクトリの下にfomodサブフォルダが生成され、そこに2つのXMLファイルがUTF-8で書き込まれます。ルートディレクトリが定義されていない限り、保存コマンドは無効のままです。

開くモードは2つ用意されています。「フォルダを開く…」はルートディレクトリを指定し、そのfomodサブフォルダからプロジェクトを読み込みます。「ファイルを開く…」はXMLファイルを直接指定します。この場合、ルートディレクトリは推測されます（ファイルはfomodサブフォルダ内にあるものとみなされます）。読み込み時、XIMOD ArchitectはファイルのエンコーディングUTF-8、およびUTF-16）を自動的に認識するため、他のツールで作成されたインストーラーを再び開くことができます。

正常に開かれた各プロジェクトは、最近使用したファイルの一覧の先頭に追加されます。この一覧はファイル → 最近使用したファイルからアクセスでき、設定の「最近使用したファイル」タブで管理できます。

2.5 初回起動

ごく最初の起動時、XIMOD Architectは英語で起動し、自動的に「設定」ウィンドウを開きます。これにより、ユーザーは国、そして言語を選択できます。

この動作は、Config.iniファイル内のFirstStartキーによって制御されます。初期設定が保存されていない間は0で、「保存」が初めてクリックされると1に切り替わります。それ以降の起動では、選択した言語で直接起動します。

ユーザーが「キャンセル」でウィンドウを閉じた場合、このキーは0のままで、次回の起動時に再びウィンドウが開きます。

2.6 フリーウィンドウとキーボード操作

いくつかのツールはフリー（独立した）ウィンドウで開きます。「国の選択」ピッカー、「プロパティ」ウィンドウ、翻訳エディター、XMLエディターです。いずれも自由に移動・サイズ変更でき、別のモニターに配置することもでき、メインウィンドウと並行して動作します。

中央配置と記憶

フリーウィンドウは初めて開くとき、XIMODのメインウィンドウの中央に配置されます。ユーザーが移動またはサイズ変更すると、その位置とサイズは閉じたとき（およびアプリケーションの終了時）に記憶され、次に開くときに復元されます。これらの値はConfig.iniの[WindowPositions]セクションに保存されます（付録Aを参照）。メインウィンドウ自体はサイズは記憶しますが位置は記憶せず、起動のたびに中央に配置されます。

Escで閉じる

Escキーは、アクティブなフリーウィンドウと、メインウィンドウのモーダルダイアログ（設定、バージョン情報、確認、検証レポート、スクリプト）を閉じます。確認では、Escは「キャンセル」として働き、操作を実行することはありません。例外が2つあります。翻訳エディター（入力中に誤って閉じるのを避けるため）とXIMODのメインウィンドウはEscでは閉じません。

キーボード操作

「プロパティ」ウィンドウのリスト、翻訳エディターの表、国ピッカーの旗グリッドは、すべてキーボードで操作できます。矢印キー（およびテンキー）、Page Up/Page Down、Home/End、Enter、Tab/Shift+Tabです。各ウィンドウに固有の詳細は該当する節で説明し、付録Bにまとめています。

フィルターのクリア

すべての「フィルター:」フィールドの右側には、ワンクリックでフィルターを空にするクリアアイコン（赤い十字）があります。

3. メニュー

メニューバーには、ファイル、編集、プロジェクト、ツール、ヘルプの5つのメニューがあります。その下のツールバーでは、日常的な操作（新規作成、開く、保存、検証、プレビュー、エクスポート）にワンクリックでアクセスできます。表示されているショートカットはグローバルでも有効です（CtrlキーはmacOSではCmdに相当します）。これらはモーダルダイアログが開いている間だけ無視されます。モーダルダイアログはウィンドウの残りの部分を暗くします。Enterで主ボタンが実行され、Escでキャンセルされます。

3.1 ファイルメニュー

コマンド	ショートカット	説明
新規作成	Ctrl+N	新しい空のプロジェクトを新しいタブで開きます（現在のタブは、まだ何も変更されていない場合にのみ置き換えられます）。
フォルダから新規…	—	MODフォルダをスキャンし、各サブフォルダが1つのオプションになるスケルトンを提案します。§ 8.4を参照してください。
フォルダを開く…	Ctrl+O	ルートディレクトリを選択し、そのfomodサブフォルダからプロジェクトを読み込みます。
ファイルを開く…	Ctrl+Shift+O	info.xmlまたはModuleConfig.xmlを直接選択します。ルートディレクトリは推測されます。fomodフォルダ内にないファイルは拒否されます。
アーカイブを開く…	—	（Nexusからダウンロードした）.zipまたは.7zのMODアーカイブを設定ディレクトリの作業フォルダに展開し、そのFOMODを開きます。すでに展開済みのアーカイブは再利用されます。.rarアーカイブは拒否されます。先に7-Zipで展開してください。アーカイブをウィンドウにドロップしても同じ動作になります。§ 8.14を参照してください。
保存	Ctrl+S	fomod/info.xmlとfomod/ModuleConfig.xmlを生成します。プロジェクトにまだルートディレクトリがない場合は、先にフォルダの指定を求められます。
名前を付けて保存…	Ctrl+Shift+S	プロジェクトの（新しい）ルートディレクトリを選択し、そこに保存します。
バックアップを復元…	—	fomod/backups以下に保存されたFOMOD XMLファイルの以前のバージョンを、現在のプロジェクトとの差異の数とともに一覧表示します。各バージョンは比較またはエディターへの復元ができます。プロジェクトにまだバックアップがない場合は無効になります。§ 8.16を参照してください。

コマンド	ショートカット	説明
FOMODをマージ…	—	別のFOMODのステップ、必須ファイル、条件付きインストールを現在のプロジェクトに追加します。ルートディレクトリが定義されていない限り無効です。§ 4.12を参照してください。
配布用アーカイブをエクスポート…	—	FOMODを書き込んだうえで、ルートディレクトリ全体を公開可能な.zipまたは.7zアーカイブに圧縮します。処理はバックグラウンドで進行し、進捗ウィンドウが表示されます。§ 4.13および§ 8.10を参照してください。
FOMODを閉じる	—	アクティブなタブを閉じます。未保存の変更がある場合は先に確認します。
すべてのFOMODを閉じる	—	すべてのタブを閉じます。未保存の変更があるタブがある場合は先に確認します。
最近使用したファイル	—	最近開いたプロジェクトを一覧表示するサブメニューです。一覧が空の場合は「(なし)」と表示されます。
終了	Ctrl+Q	アプリケーションを終了します。未保存のプロジェクトは一覧表示され、先に保存できます。設定は終了時に保存されます。

3.2 編集メニュー

コマンド	ショートカット	説明
元に戻す	Ctrl+Z	プロジェクトへの最後の変更を元に戻します（同じフィールドへのキー入力はまとめられます）。元に戻すものがない場合は無効です。
やり直し	Ctrl+Y	最後に元に戻した変更を再適用します（Ctrl+Shift+Zでも動作します）。
切り取り	Ctrl+X	ツリーで選択したステップ、グループ、オプションをプロジェクトから取り除き、クリップボードに保持します。編集 → 元に戻すで元の位置に戻せます。§ 8.30を参照してください。
コピー	Ctrl+C	選択したステップ、グループ、オプションを、その内容すべてとともにクリップボードへ（さらにシステムのクリップボードへも）コピーします。§ 8.30を参照してください。
貼り付け	Ctrl+V	クリップボードの内容を選択項目の隣に挿入します。ステップは選択したステップの後に、グループは選択したステップの中に、オプションは選択したグループの中に入ります。§ 8.30を参照してください。

3.3 プロジェクトメニュー

編集中のFOMODに作用するすべての操作です。

コマンド	ショートカット	説明
FOMODを検証	—	プロジェクトと、ModuleConfig.xmlのModConfig 5.0スキーマへの適合性を確認し、参照されているファイル、宛先の衝突、画像を（バックグラウンドで）検証したうえで、詳細なレポートを表示します。§ 4.8を参照してください。
インストーラのプレビュー…	—	MODマネージャーが表示するようにインストーラーをシミュレートします。§ 4.11を参照してください。
欠落ファイルを検索…	—	プロジェクトが使用しているファイル、フォルダ、画像のうち、もう存在しないものを一覧表示し、その移動先を提案します。§ 8.32を参照してください。
条件エディター…	—	各条件フラグについて、それが設定・使用されている場所（クリック可能）を表示し、フラグの名前変更や全体からの削除を行えます。また、選択中のノード向けの条件ビルダーも備えています。§ 8.7および§ 8.22を参照してください。
FOMODを翻訳…	—	既存のインストーラー（アクティブなプロジェクト、フォルダ、またはアーカイブ）のXML構造を変えずに翻訳するウィンドウを開きます。§ 8.13を参照してください。
比較…	—	プロジェクトを別のFOMODと比較し、変更点を一覧表示します。§ 8.6を参照してください。
テンプレート…	—	選択中のステップを再利用可能なテンプレートとして保存したり、保存済みのテンプレートを挿入したりします。§ 8.3を参照してください。
プリセットウィザード…	—	現在のプロジェクトに「プリセット＋カスタマイズ」型のインストーラーを生成します。プリセットを並べたインストールモードのページ、必要に応じて表示される機能ページ、動的なデフォルト、条件付きインストールで構成されます。§ 7.5および§ 8.29を参照してください。
XMLエディター	—	info.xmlとModuleConfig.xmlにアクセスするためのサブメニューです。§ 4.10を参照してください。

3.4 ツールメニュー

特定のプロジェクトに結び付かない設定やユーティリティです。

コマンド	ショートカット	説明
設定…	Ctrl+,	設定ウィンドウ（国、言語、テーマ、フォントなど）を開きます。§ 4.5を参照してください。

コマンド	ショートカット	説明
ツリーの色…	—	プロジェクトツリーの行の種類（ステップ、グループ、オプション…）ごとに、テーマ別に1色を選びます。§ 8.27を参照してください。
保存前スクリプト…	—	保存前に実行されるスクリプトを編集します。§ 4.6を参照してください。
保存後スクリプト…	—	保存後に実行されるスクリプトを編集します。§ 4.6を参照してください。
プラグインレポート…	—	プロジェクトがインストールするすべてのプラグインファイル（.esp、.esm、.esl）の表です。種類、lightフラグ、マスター（欠落しているものを強調表示）、light plugin上限に対する新規レコード数、ESL適格性を示します。§ 8.18を参照してください。
アーカイブの内容…	—	ベセスダの.bsaまたは.ba2アーカイブに格納されたファイル（形式、サイズ、圧縮）を検索フィールド付きで一覧表示します。§ 8.24を参照してください。
Nexus用説明文…	—	プロジェクトからNexus Modsのページ用テキスト（BBCodeまたはMarkdown）を生成します。要件、インストールオプション、インストールに関する注意、以前のバージョンとの変更履歴を含みます。コピーまたは保存できます。§ 8.21を参照してください。
プロジェクトの文字列…	—	プロジェクトのすべての名前と説明の表です。検索、検索／置換、重複検出を備え、テキストはその場で編集できます。§ 8.23を参照してください。
インターフェースを翻訳…	—	インターフェース翻訳エディターを開きます。§ 4.9を参照してください。
プロパティ…	—	国／言語データベースの読み取り専用エクスプローラーを開きます。§ 4.14を参照してください。
ゲーム / カテゴリ…	—	ゲームエディターを開きます。Categories.jsonを手作業で編集することなく、ゲームとそのNexusカテゴリ（名前、slug、基本マスター、ライトプラグインの上限）を追加、修正、削除できます。§ 8.26を参照してください。

3.5 ヘルプメニュー

コマンド	ショートカット	説明
ユーザーマニュアル	F1	本マニュアル（PDF）を、アプリケーションに付属するManualsフォルダから、インターフェースの言語で開きます。

コマンド	ショートカット	説明
更新を確認…	—	公開されている最新バージョンを問い合わせ、ダウンロードリンクを提示します。
バージョン情報	—	「バージョン情報」ウィンドウ（名前、バージョン、ライセンス）を表示します。§ 4.7を参照してください。

4. 機能の詳細

4.1 Mod情報タブ

このタブには、MODのメタデータとルートディレクトリの選択がまとめられています。

ワークスペース — 「ルートディレクトリ:」の行には作業フォルダが表示されます（定義されていない場合は「(ディレクトリを選択)」と表示されます）。「参照...」ボタンで選択できます。

メタデータのフィールドは次のとおりです。

フィールド	役割
MOD名	インストーラーの表示名（必須）。
作者	MODの作者名。
バージョン	バージョン番号（新規プロジェクトではデフォルトで1.0.0）。
ゲーム名	対象ゲーム。Categories.jsonから取得した一覧から選択します。
カテゴリ	MODのカテゴリ。選択したゲームによって異なります。該当しない場合はデフォルトの一覧が提示されます（アニメーション、防具、オーディオ、テクスチャ、武器など）。
ウェブサイト	MODのページのアドレス。
ヘッダー画像	インストーラーの上部に表示される画像。「参照...」ボタンと「クリア」ボタンがあり、プレビューが表示されます。パスはルートディレクトリからの相対パスで保存されます。
説明	MODの紹介テキスト（複数行の入力欄）。

> 注 —
> いずれかのフィールドを変更すると、ステータスバーでプロジェクトが「変更あり」として表示されます。

Nexusリンク — 「ゲーム名」セレクターの横にある「Nexus ↗」ボタンをクリックすると、選択したゲームのNexus Modsページ（その「スラッグ」から生成）がブラウザで直接開きます。選択したゲームにスラッグがある場合にのみ有効です。

「ゲーム名」と「カテゴリ」の一覧は、外部ファイルassets/data/Categories.jsonから取得され、再コンパイルせずに編集できます。各ゲームは、識別子、名前、Nexusの「スラッグ」、およびカテゴリの一覧で記述されています。

```
{
  "games": {
    "skyrimSpecialEdition": {
      "name": "The Elder Scrolls V: Skyrim Special Edition",
      "nexusSlug": "skyrimspecialedition",
      "categories": ["Animation", "Armour", "Audio", "…"]
    }
  }
}
```

4.2 インストール手順タブ

ここはインストールウィザードの中核です。3階層の構造（ステップ → グループ → オプション）に基づいており、これは左側のプロジェクトツリーの形そのままです。「インストールステップ」の横にある+ボタン（デフォルトで「ステップN」と命名）または空のインスペクターからステップを追加します。ツリーでステップ、グループ、オプションを選択すると中央で編集できます。ステップエディターには、名前（編集可能）、「ステップを削除」ボタン（確認を求める唯一の削除操作です。それ以外はすべてCtrl+Zで元に戻せます）、表示条件、ページの宛先、「グループを追加」ボタン付きのグループ一覧が表示されます。グループエディターには、名前、選択タイプ、グループの宛先、「オプションを追加」ボタン付きのオプション一覧が表示されます。オプションエディターについては後述します。

グループと選択タイプ

各グループには、名前と、エンドユーザーがチェックできるオプションの数を決める選択タイプがあります。利用可能な5つのタイプは次のとおりです。

タイプ	意味
正確に1つ（必須） — SelectExactlyOne	ユーザーはちょうど1つのオプションを選ばなければなりません。
多くても1つ — SelectAtMostOne	多くても1つのオプション（または選ばないこと）。
任意の数 — SelectAny	任意の数のオプション（デフォルト値）。
すべて（選択なし） — SelectAll	すべてのオプションが選択されます。
少なくとも1つ — SelectAtLeastOne	少なくとも1つのオプション。

ステップエディターの「グループを追加」ボタン（またはステップの右クリックメニュー）で、デフォルトは「任意の数」タイプのグループを作成します。グループの右クリックメニューまたはグループエディターの「削除」でグループを削除します。

並べ替え —
すべてを作り直すことなく表示順を調整できます。ツリーでノードをドラッグして配置したい場所にドロップするか（オプションを別のオプションの前またはグループ上へ、グループを別のグループの前またはステップ上へ、ステップを別のステップの前へ）、ノードの右クリックメニューの「前に移動」／「後に移動」を使います。「複製」はステップ、グループ、オプションを、その内容すべてとともにコピーします。すべての変更は編集 →
元に戻す（Ctrl+Z）で取り消し、編集 →
やり直し（Ctrl+Y）で再適用できます。フィールドへの連続したキー入力は1つの操作として扱われます。切り取り、コピー、貼り付け（同じ右クリックメニュー、編集メニュー、またはCtrl+X / Ctrl+C / Ctrl+V）を使うと、ノードをプロジェクト内の別の場所や、開いている別のFOMODへ移動できます（§ 8.30）。

オプションとデフォルトタイプ

オプション（FOMOD形式の<plugin>要素）は、ユーザーに提供される選択肢です。名前、説明、画像（プレビュー、「参照…」ボタン、「クリア」ボタン）、およびデフォルトタイプを持ちます。

タイプ	意味
オプション（任意） — Optional	任意のオプション（デフォルト）。
必須 — Required	常にインストールされるオプション。
推奨 — Recommended	推奨されるが任意のオプション。
使用不可 — NotUsable	使用できないオプション。
使用できる可能性あり — CouldBeUsable	条件次第で使用できる可能性のあるオプション。

プラグインファイル

「ソース」セクションには、オプションによってインストールされるファイルとフォルダが、4列の表（タイプ、ソース、宛先、優先度）として表示されます。「ファイルを追加」、「フォルダを追加」、「削除」ボタンで一覧を管理します。「宛先」列は編集可能です。セルをクリックして、対象をインストールするパスを、ゲームのDataフォルダからの相対パスで入力します。デフォルトは空で、その場合はファイル（またはフォルダの内容）がDataのルートにインストールされます。これは、ソースフォルダがすでにゲームの構造を再現している場合（例えば、それ自体がmeshes、textures、SKSEなどを含むフォルダ）に適した選択です。対象が異なる場合にのみ、SKSE\Plugins\MyMod.iniのように明示的なパスを入力してください。ファイルはルートディレクトリの下に置く必要があります。オプションを選択した状態で、ファイルエクスプローラー（Windowsのエクスプローラー、Finder…）からファイルやフォルダをXIMODのウィンドウへ直接ドラッグすることもできます。それらはすぐにそのオプションのソース表に追加されます（§ 8.5参照）。ダイアログは前回選んだフォルダで開くため、同じ階層のフォルダにはワンクリックで移動できます（§ 8.28）。「ディスク上で名前を変更…」は、選択したファイルまたはフォルダの名前をMODフォルダ内で変更し、それへの参照をすべて更新します（§ 8.31）。

グループ／ページの宛先

各ファイルを編集せずに複数のオプションを同じ場所へ送るには、一括宛先コントロールを使います。グループのエディターでは、「グループ全体の宛先」フィールドとその「このグループのすべてのオプションに適用」ボタンで、そのグループ内のすべてのオプションのすべてのファイルに1つのインストール先を一度に設定します。ステップレベルでは、折りたたみ可能な「インストール先（ページ全体）」パネルが、そのページのすべてのグループにまたがるすべてのオプションに対して同じことを行います。ステータスメッセージが、更新されたファイル数を確認します。オプションに実際にファイルが含まれるまでボタンは無効のままで、後から個々の宛先をオプションのソース表で微調整することもできます。

条件フラグ

「フラグ名:」セクションでは、オプションにフラグ（名前＝値のペア）を関連付けられます。エンドユーザーがオプションを選択すると、そのフラグが設定されます。これらのフラグはその後、ステップの表示可否、オプションのタイプ（依存関係パターン）、または条件付きファイルのインストールを制御できます。「フラグを追加」および「フラグを削除」ボタンで一覧を管理します。

オートコンプリート —
フラグや依存関係の名前・値フィールドでは、入力中にプロジェクト内に既に存在する要素から抽出した候補メニューが表示されます。これにより、マッチングを無言で壊してしまう入力ミスを防げます。ファイル依存関係では、値としてActive／Inactive／Missingの状態が候補に表示されます。候補は強制ではなく、新しい値を自由に入力することもできます。

オプション依存関係。フラグの下にあるこのセクションは、オプションのタイプを動的にします。上部のデフォルトタイプは、どのパターンも一致しない場合に適用されます。各パターンは、その条件が満たされたときにオプションに別のタイプを与えます。新しいパターン／パターンを削除で一覧を管理します。選択したパターンについて、パターン演算子（And／Or）と適用するパターンタイプ（Optional、Required、Recommended、NotUsable、またはCouldBeUsable）を選び、その条件を追加します。条件にはタイプ（FileまたはFlag）、File/Flagの名前（オートコンプリート付き）があり、ファイルの場合はState（Active、Inactive、またはMissing）、フラグの場合は期待値を持ちます。例えば、デフォルトタイプをNotUsableにし、ファイルDLCCoast.esmがActive状態であることを条件とするOptionalタイプのパターンを追加すると、Far Harborがインストールされている場合にのみオプションが選択可能になります。

ステップの表示条件。各ステップは条件付きで表示できます。ステップ名の下にある折りたたみ可能な「表示条件」セクションには、ステップの表示ルールが含まれます。演算子（And／Or）を設定し、同じ方法でFileまたはFlagの条件を追加します。少なくとも1つの条件が含まれる場合、ステップはそれらが満たされたときにのみウィザードに表示されます。例えば、DLCCoast.esmがActiveのときにのみ表示されるパッチのステップです。常にステップを表示するには空のままにします。

4.3 必須のインストールタブ

このタブには、ユーザーの選択に関係なく常にインストールされるファイルとフォルダが一覧表示されます。オプションのファイルセクションと同じ4列の表（タイプ、ソース、宛先、優先度 —
同じく編集可能な「宛先」列付き）と、同じ「ファイルを追加」、「フォルダを追加」、「削除」ボタンがあります。表の上にある「宛先」フィールドとその「すべての必須ファイルに適用」ボタンを使うと、すべての必須ファイルに1つのインストール先を一度に設定できます。これは§
4.2のグループ／ページの宛先コントロールに相当するもので、各宛先は後から微調整することもできます。

4.4 条件付きインストールタブ

このタブはパターンをまとめます。各パターンは、その条件が満たされたときにのみ一連のファイルをインストールします。「パターンN」のタブが並んだ行で、パターンの追加（「＋」）や削除（「－」）を行えます。

選択したパターンについて：

- 演算子 — AndまたはOr：パターンの依存関係をどのように組み合わせるか。
- 依存関係 — Flag（[Flag] 名前 = 値）またはFile（[File] 名前 (状態)）タイプの条件の一覧。「依存関係を追加」および「依存関係を削除」ボタンで一覧を管理します。
- ファイル — 条件が満たされたときにインストールされるファイルの4列の表。その上にある「宛先」フィールドとその「このセットのすべてのファイルに適用」ボタンを使うと、パターンのすべてのファイルに1つのインストール先を一度に設定できます。プリセットウィザード（§ 8.29）がオプションのファイルをここへ移した後に便利です。

注 — 「プラグイン依存関係」。プラグイン依存関係とは、ファイルがゲームのプラグイン（.esm/.esp）であるFile依存関係のことです。名前フィールドには、ゲーム内に表示されるとおりのプラグインのファイル名（例：Fallout4.esm、DLCCoast.esm、または別のMODの.esp）を入力します。MODの表示名ではありません。状態を選択します：Active（プラグインが存在し有効 — 通常の選択）、Inactive（存在するが無効）、Missing（存在しない）。File依存関係は3か所で使用できます：ファイルを自動的にインストールする（条件付きインストールタブ）、ウィザードのステップを表示・非表示にする（ステップのvisible条件）、オプションの状態を変更する — 例えば、DLCが有効なときにRecommendedにし、必須プラグインが欠落しているときにNotUsableにします。

4.5 「設定」ウィンドウ

ツール → 設定（Ctrl+,）からアクセスできます。「一般」と「最近使用したファイル」の2つのタブがあります。設定はConfig.iniに保存されます。

「一般」タブは2つの列で構成されています。

左の列 — 国旗とその名前：

項目	役割
旗	画像をクリックすると国セレクターが開きます（下記参照）。
国名	選択した言語で書かれた国名（エンドニム）。編集不可のフィールドで、2行で表示されます。

右の列 — インターフェース設定：

設定項目	説明

設定項目	説明
言語	インターフェースの言語。すぐに適用されます。国が選択されるまでこのフィールドは無効のままで、選択後はその国で話されている言語が提示されます。
テーマ	ダーク、ライト、またはシステム。
フォントサイズ	インターフェースのテキストのサイズ。
改行を処理する	説明文中の\nシーケンスを解釈します。
最近使用したファイルの最大数	保持するエントリの数。
ウィンドウの幅／高さ	メインウィンドウの保存されたサイズ。

国セレクター —
旗をクリックすると「国の選択」ウィンドウが開き、244の旗がグリッド状に表示されます。縦横にスクロールできます。これはフリーウィンドウで、開くときにメインウィンドウの中央に配置されます（§
2.6を参照）。「フィルター:」フィールドでは、フランス語名、英語名、または国コードで一覧を絞り込めます。フィールドの右側にあるクリアアイコン（赤い十字）で、ワンクリックでフィルターを空にできます。各旗の下に国名が表示されます。

グリッドはキーボードでも操作できます。矢印キー（またはテンキーの8/2/4/6）で選択枠を移動し、Page Up/Page Downで1画面スクロール、Home/Endで最初と最後の旗へジャンプ、TabとShift+Tabで1つ先・前の旗へ移動、Enterでハイライトされた国を確定、Escでウィンドウを閉じます。詳細は付録Bに記載しています。

選択した国によって、提示される言語の一覧が決まります。XIMODはCountryLanguages.jsonファイルを利用しており、このファイルには各国で話されているすべての言語が列挙されています。パプアニューギニアでは最大844に上るため、ドロップダウンリストはスクロールします。

バージョン2で3つの設定が追加されました。保持するバックアップ数（各保存時にfomod/backups以下に保持するFOMOD XMLの以前のバージョンの数。0 = なし、デフォルト10）、N分ごとの復旧コピーの自動保存（この間隔で変更済みの各プロジェクトの復旧コピーが設定フォルダに書き込まれ、異常終了した後の次回起動時にのみ提示されます。0 = オフ、デフォルト5）、そしてプラグインのマスターを条件として追加する（§
8.17を参照。デフォルトでオン）です。

4.6 保存前・保存後スクリプト

XIMOD
Architectは、保存時にスクリプトを自動的に実行できます。FOMODファイルが書き込まれる直前に起動される保存前スクリプトと、その直後に起動される保存後スクリプトです。これらは、MOD生成に付随する作業を自動化するのに役立ちます。例えば、保存前に古いアーカイブを整理したり、ビルド番号を加算したり、FOMODの書き込み後にMODを公開可能なアーカイブへ自動的にパッケージ化したりできます。

これらはツール → 保存前スクリプト…とツール → 保存後スクリプト…で編集します。ウィンドウには、ヘルプテキスト、利用可能なマクロの一覧、等幅フォントの入力エリア、「保存」ボタンと「キャンセル」ボタンが表示されます。両スクリプトの内容はConfig.ini（PreSaveScriptキーとPostSaveScriptキー）に保存されるため、セッションをまたいで記憶されます。

仕組み：保存時、XIMODはまずマクロをその値に置き換え、結果を一時ファイルに書き出してから、システムのインタープリターで実行します。Windowsではcmdが起動する.batコマンドファイル、LinuxとmacOSではshが起動する.shシェルスクリプトです。一時ファイルはその後削除されます。保存前スクリプトはXMLファイルが書き込まれる前に、保存後スクリプトはその直後に実行されます。空のスクリプトは単に無視されます。

実行前に、スクリプトのテキスト内で次のマクロが置き換えられます。

マクロ	置換される値
$MODNAME$	MOD名。
$MODAUTHOR$	作者名。
$MODVERSION$	MODのバージョン。
$MODROOT$	ルートディレクトリのパス。
$DATE$	現在の日付（YYYY-MM-DD）。
$TIME$	現在の時刻（HH:MM:SS）。
$RANDOM$	乱数。

例 — 保存前スクリプト（Windows）

保存前に、MODのルートに既にある.rarアーカイブを削除し、きれいなフォルダから始めます。

```
del "$MODROOT$\*.rar" /q
```

例 — 保存後スクリプト（Windows）

FOMODの書き込み後、WinRARを使って、MOD名とバージョンにちなんだ名前の.rarアーカイブにMODの内容全体を自動的にパッケージ化します。

```
d:
cd "$MODROOT$"
"C:\Program Files\WinRAR\rar.exe" a -r -ep1 "$MODNAME$_$MODVERSION$.rar" *
```

各行の説明：d:でドライブD:に切り替え、cd "$MODROOT$"でMODのルートに移動し、WinRARがアーカイブを作成します（-rは再帰、-ep1は親ツリーを再現しないため）。名前は例えばDawnlight_1.2.0.rarです。

Linux／macOSでの同等の処理

LinuxとmacOSでは、スクリプトはシェルスクリプトとして解釈されます。同じ保存後パッケージ化は、例えばzipで次のように書きます。

```
cd "$MODROOT$"
zip -r "$MODNAME$_$MODVERSION$.zip" .
```

> 注 —
> これらのスクリプトは、あなたのユーザー権限で実際のシステムコマンドを実行します。delやrmのようなコマンドは破壊的です。実際のMODで使う前に、使い捨てのプロジェクトでテストしてください。

4.7 「バージョン情報」ウィンドウ

ヘルプ →
バージョン情報で開くこのウィンドウには、アプリケーション名、バージョン番号、簡単な説明（「ベセスダのゲームMOD向けFOMODインストーラーを作成するためのクロスプラットフォームツール。」）、および「MITライセンスのもとで提供」という記載が示されます。「OK」ボタンで閉じます。

4.8 FOMODファイルの生成

保存時、XIMOD
Architectはプロジェクトを2つの標準化されたXMLファイルに変換し、fomodサブフォルダにUTF-8で書き込みます。

書き込み前にプロジェクトが確認されます。異常（MOD名の欠落、ステップや必須ファイルがない、名前のないステップやグループ、オプションのないグループなど）が含まれる場合、ウィンドウにそれらがすべて一覧表示され、「それでも保存」が提示されます。保存がブロックされることはなく、作業途中のプロジェクトを意図的に保存できます。

スキーマ検証 — このプロジェクト確認に加えて、生成されたModuleConfig.xmlをModConfig 5.0スキーマに照らした構造チェックがあります。これはRustで記述されています（外部のXSDエンジンを使いません）。要素の階層と出現回数、必須属性、列挙値（選択タイプとプラグインタイプ、ファイル状態、演算子）を確認します。各異常は行と列で特定されます。この検証には3つの方法でアクセスできます。完全なレポートを表示するプロジェクト →
FOMODを検証コマンド、ドキュメントが整形式になり次第XMLエディター内でリアルタイムに、そして保存確認ウィンドウに統合された形で、です。

info.xmlには、<fomod>ルートの下にメタデータが含まれます。名前（Name）、作者（Author）、バージョン（Version）、ウェブサイト（Website）、説明（Description）、カテゴリ（Groups）、ゲーム（Game）です。Game要素はXIMOD固有の拡張で、MODマネージャーには無害に無視されますが、再読み込み時に正しいカテゴリ一覧を復元するのに役立ちます。

ModuleConfig.xmlには、<config>ルートの下にインストールが記述されます。モジュール名、ヘッダー画像、必須ファイル（requiredInstallFiles）、ステップ（installSteps → group → plugin）、条件付きインストール（conditionalFileInstalls）です。各pluginは、その説明、画像、条件フラグ、ファイル、タイプを記述します。依存関係はAndまたはOr演算子で組み合わされます。

4.9 翻訳エディター

ツール →
インターフェースを翻訳…からアクセスできるこのツールでは、ファイルを手で編集せずにインターフェースを翻訳したり、既存の翻訳を修正したりできます。

ウィンドウのヘッダー

フィールド	役割
旗	国の選択。翻訳可能な言語の一覧が決まります。
国のエンドニム名	翻訳先の言語での国名。選択したフォントで表示されます。
言語のエンドニム名	その言語自体での言語名（例：「Français」）。
作者	翻訳者の名前。
表示する言語	2列目に表示される参照言語（すでにmain.ftlファイルを持つものの中から選択）。
翻訳する言語	翻訳先の言語。その国の言語の中から選択します。
フォント	この言語に使用するフォント。「参照…」ボタンで、assets/fontsを基点とするファイルセレクターが開きます。
Google Fonts	ブラウザでfonts.google.comサイトを開きます。

重要な制約：フォントはassets/fontsフォルダにインストールされている必要があります。別の場所にあるフォントは拒否されます。プログラムはそのフォルダからの相対パスでフォントを読み込むため、次回起動時に見つけられないからです。フォントがない場合は、Google Fontsからダウンロードし、そこにコピーしてから選択する必要があります。

同様に、翻訳できるのはISO 639-3コードを持つ言語のみです。このコードが翻訳フォルダの名前となり、言語をそのフォントと参照データに結び付けます。そうでない場合、保存ボタンは無効になります。

翻訳表 — 3つの列：

1列目	2列目	3列目
キー（編集不可）	表示する言語でのラベル	翻訳の入力フィールド

トークンの保護：翻訳してはならない要素（$MODNAME$などのマクロや、{ $num }などの変数）は、2列目にそのまま表示されることはなく、自動的に保持されます。

- テキストの先頭または末尾にあるトークン：非表示にされ、保存時に再挿入されます。
- テキストの途中にあるトークン：[[1]]、[[2]]…といったコンパクトなマーカーに置き換えられ、保持・再配置されます。

アプリケーション名（キーapp-title）は意図的に除外されており、翻訳できません。

キーボード操作 — 表はマウスなしで操作できます（付録Bを参照）。上／下の矢印とPage Up/Page Downで行をスクロールし、Home/Endで最初と最後へジャンプ、Enterでハイライトされた行の入力フィールドにカーソルを置きます。編集を始めると、Tabで次のフィールドへ、Shift+Tabで前のフィールドへ移動するので、キーボードから離れずに次々と翻訳を入力できます。Page Up/Page Downで移動する行数は、ウィンドウの高さに合わせて自動的に調整されます。

保存 — 「保存」ボタンはファイルassets/locales/<言語>/main.ftlを書き込みます。先頭には、Fluentのコメントとして書かれたメタデータブロックが付きます（そのためファイルは完全に有効なままです）。

```
# XIMOD Architect - translation metadata
# @language = fra
# @font = Noto_Sans/static/NotoSans-Regular.ttf
# @langname = Français
# @author = Patrice
```

ヘッダーには言語のみが記述され、国の情報は含まれません。同じ言語が複数の国で話されているためです（ドイツ語はドイツでもスイスでも話されます）。国は選択した旗によって決まり、XIMODはCountries.jsonとCountryLanguages.jsonから国 → 言語の対応を処理します。

エンドニムフィールドの入力 —
国（旗）と言語が選択されると、XIMODは2つのフィールドに入力します。

- 国のエンドニム名：選択した国について常にCountries.jsonから解決されるため、旗に従います（ドイツ語ではドイツに「Deutschland」、スイスに「Schweiz」が表示されます）。
- 言語のエンドニム名：ファイルのヘッダー（# @langname）がある場合はそこから取得されます。それがない場合（ヘッダーのない古いファイルや、まだ翻訳されていない言語）は、Languages.jsonから取得されます。

この2つのフィールドは編集可能なままです。ユーザーはそこで修正でき、修正内容は保存時に参照ファイルへ反映されます。

保存すると参照データが更新されます。Languages.json内の言語のエンドニムとフォント、Countries.json内の国のエンドニム（国／言語のペアについて）です。その国についてその言語がまだ存在しなかった場合は、エントリが作成されます。

ヘッダーはもはや国を持たないため、XIMODはmain.ftlとともに、タブ区切りの小さなファイルcountryEndonyms.tsvを書き込みます。これは、言語についての国のエンドニムを「国コード, 言語コード, エンドニム」の形式で国ごとに1行ずつ記録します。言語フォルダのみが利用可能な場合に国のエンドニムを復元できるのは、このファイルのおかげです。

翻訳の送信 —
「送信…」ボタンは、翻訳ファイル（main.ftl、その言語フォルダ内）と、存在する場合は付随するcountryEndonyms.tsvファイルを含むZIPアーカイブを構築します。XIMODはアーカイブのフォルダを開いてから、あらかじめ入力されたメールを用意します。言語固有の情報（言語、フォント、言語のエンドニム、作者）はmain.ftlのヘッダーに、国のエンドニムはcountryEndonyms.tsvに含まれます。Languages.jsonもCountries.jsonも添付されません。実際に使用されたコードのみが送信され、ISO 639-3コードの利用規約が尊重されます。ファイルの添付と送信はユーザー自身が行います。明示的な操作なしにデータがマシンから出ることはありません。

> 注 —
> 翻訳が完了したときにのみパッケージを送信してください。翻訳されずに残ったキーは空の値で保存され、インターフェースにテキストなしで表示されてしまいます。

4.10 XMLエディター

プロジェクト → XMLエディター → info.xmlまたはModuleConfig.xmlからアクセスできるこの上級者向けツールでは、生成されたXMLを表示し、直接編集できます。

独立したウィンドウで開き、自由に移動・サイズ変更でき、別のモニターに配置することもできます。

2つのモード

- *読み取り専用（デフォルト）：XMLはシンタックスハイライトとぶら下げインデントで表示されます。長すぎる行は折り返され、Notepad++のように行頭の下に揃えられます。メインウィンドウは引き続き使用できます。*
- *編集（「変更」ボタン）：テキストが編集可能になり、唯一のソースを正とするためにメインウィンドウのタブがロックされます。*

シンタックスハイライト：タグ、属性名、引用符付きの値、コメント、宣言、テキストが、ライトまたはダークテーマに適した色で区別されます。

行番号：ガターに番号が表示されます。折り返しの継続行は空白のままです。

リアルタイム検証：入力中、XMLは継続的にチェックされます。緑のインジケーターが整形式のドキュメントであることを示します。エラーの場合は、赤いメッセージが行と列を特定し、問題のあるトークンに下線が引かれます。

変更の適用：「適用」ボタンはテキストを再解析し、プロジェクトを更新します。エラーの場合、変更は拒否され、テキストは編集可能なままです。「キャンセル」ボタンはプロジェクトからテキストを再生成します。

4.11 フォント管理

XIMOD
Architectには、一覧にある言語のすべての文字体系をカバーするNotoフォントのセットが付属しており、assets/fontsに置かれています。

これらのフォントは動的に読み込まれます。いつでも、実際に必要なものだけ、すなわちインターフェース言語のフォントと、選択した国の言語のフォントがインストールされます。負荷は低く保たれ、ほとんどの国では単一のフォントで済みます。

この仕組みにより、非ラテン文字（タイ語、ビルマ語、アムハラ語、モン語など）が正しく表示されます。そうでなければ空の四角として表示されてしまいます。これはインターフェースだけでなく、「プロパティ」ウィンドウや翻訳エディターにも適用されます。そこで表示されるすべての言語のフォントが読み込まれるため、名前やラベルが四角として表示されることはありません。

XIMOD
Architectには32のインターフェース翻訳が付属しており、それぞれが非ラテン文字を含む文字列一式を完全にカバーしています。日本語（jpn）、簡体字中国語（zho）、韓国語（kor）、ロシア語（rus）です。英語（eng）とフランス語（fra）は手作業で保守される参照翻訳です。その他は、ネイティブスピーカーが翻訳エディターで見直せる貢献によるものです。

各言語に関連付けられたフォントはLanguages.jsonに記載されており、翻訳エディターから変更できます。

4.12 2つのFOMODの統合

ファイル →

FOMODをマージ…コマンドでは、2つのインストーラーのインストール内容を結合して組み合わせられます。現在開いているプロジェクトが受け手となるため、あらかじめ作成または開いておく必要があります。そうでない場合、このコマンドは無効のままです。

クリックするとファイルセレクターが開きます。統合するFOMOD（提供元）のModuleConfig.xmlファイルを選択します。そのステップ、必須ファイル、条件付きインストールが、受け手プロジェクトのそれらの末尾に追加されます。受け手プロジェクトのメタデータ（名前、作者、バージョン、カテゴリ、ヘッダー画像）は保持されます。インポートされるのはインストール内容のみです。

統合後、プロジェクトは変更済みとしてマークされます。結果をFOMODファイルに書き込むため、忘れずに保存してください（Ctrl+S）。統合を繰り返して、複数のFOMODを次々と組み込めます。

> 注 —
> 読み込まれるのは提供元のModuleConfig.xmlファイルのみです。その独自のメタデータ（info.xml）はインポートされません。統合後は、結合されたステップ全体で優先度と条件に矛盾がないか確認してください。

4.13 配布用アーカイブのエクスポート

ファイル →

配布用アーカイブをエクスポート…コマンドは、アップロード可能なアーカイブ（Nexus Modsやその他のMODマネージャー向け）を一度の操作で生成します。XIMODはまずFOMOD（fomod/info.xmlとfomod/ModuleConfig.xml）をルートディレクトリに書き込み、その後フォルダ全体をZIP形式で圧縮します。ルートディレクトリが定義されていない限り、このコマンドは無効です。

セレクターは「<名前>-<バージョン>.zip」の形式のデフォルト名を提示します（禁止文字は除去されます）。保存前に変更できます。その後、ステータスバーに書き込まれたファイル数とアーカイブのパスが表示されます。

アーカイブの構成 —

fomod/サブフォルダとMODファイルは、MODマネージャーが期待するとおり、アーカイブのルートに配置されます。内部パスにはスラッシュ「/」が使われます（クロスプラットフォーム互換性のため）。

> 注 —
> 不要ファイル（.git、.DS_Store、Thumbs.db、desktop.iniなど）とアーカイブ自体は自動的に除外されます。したがって、エクスポート前に、すべてのMODファイルが確かにルートディレクトリの下にあることを確認してください。

4.14 「プロパティ」ウィンドウ

ツール →

プロパティ…で開くこの独立したウィンドウ（移動・サイズ変更可能で、別のモニターに配置

することもできます）は、埋め込まれた国／言語データベースの読み取り専用エクスプローラーを提供します。ここでデータベースを変更することはできません。参照用のツールです。

国タブ：選択した国について、ウィンドウはその旗、フランス語名と英語名、公用語（エンドニム、フォント、ISO 639-3／639-1コード付き）、およびそこで話されているすべての言語の一覧を表示します。

言語タブ：選択した言語について、ウィンドウはそのISOコードとフォントを表示し、その言語が話されている国の逆引きを行います。「使用される国」の一覧は、各国を設定で選んだ言語（旗 → 国 → 言語）で表示します。その言語がその国で公用語である場合はCountries.jsonのその言語でのエンドニム（イタリア語話者のユーザーには「Confederazione Svizzera」）、そうでなければCountryNames.jsonの翻訳名（ドイツ語話者のユーザーには「Südafrika」）、最後の手段として英語名が使われます。

キーボード操作 ― 国の一覧と言語の一覧は、どちらもキーボードで操作できます（付録Bを参照）。上／下の矢印、Page Up/Page Down（その刻み幅はウィンドウの高さに合わせて調整されます）、Home/End、そしてハイライトされた項目を選択するEnterです。TabとShift+Tabは、旗に移ることなく一覧内で選択を移動します。各「フィルター:」フィールドには右側にクリアアイコン（赤い十字）があり、ウィンドウはEscで閉じます。

4.15 コマンドラインモード（CLI）

引数を付けて起動すると、XIMOD Architectはウィンドウを開かず、要求されたコマンドを実行してから終了コード（成功時は0、失敗または検証異常時は0以外）で終了します。このモードにより、1つまたは複数のFOMODの再ビルドとパッケージ化を、スクリプトや継続的インテグレーションのパイプラインで自動化できます。

> 注 —
> Windowsでは、実行ファイルが「windows」アプリケーションとしてコンパイルされているため、メッセージを書き込むために起動元のコンソールにアタッチします（AttachConsole経由）。出力を見るには、コマンドプロンプト（cmdまたはPowerShell）から起動してください。CLIコマンドをダブルクリックしても何も表示されません。

利用可能なコマンドは次のとおりです。

コマンド	効果
ximod-architect validate <root>	ディレクトリのFOMODを検証します（プロジェクトの確認とModConfig 5.0スキーマへの適合性）。各異常が特定されます。異常が見つかった場合は0以外の終了コードになります。
ximod-architect build <root>	既存のプロジェクトからfomod/info.xmlとfomod/ModuleConfig.xmlを（再）書き込みします。
ximod-architect package <root> [-o FILE]	FOMODを書き込んだうえで、.zip配布アーカイブを構築します。-oオプションで出力パスを指定できます。指定しない場合、アーカイブは現在のディレクトリに作成されます。
ximod-architect help	ヘルプ（コマンド一覧）を表示します。
ximod-architect version	バージョン番号を表示します。

ここで<root>は、MODのルートディレクトリ（fomodサブフォルダを含む、または今後含むフォルダ）を指します。

4.16 更新の確認

デフォルトでは、XIMOD Architectは起動時に1日1回、新しいバージョンが利用可能かどうかを確認します。新しいバージョンがある場合、メニューバーの下に控えめなバナーが表示され、ダウンロード用のリンク（NexusとGitHub）が示されます。ヘルプ → 更新を確認からいつでも確認することもできます。これは設定ウィンドウ（「起動時に更新を確認する」）でオフにできます。

プライバシー：この確認は、アプリケーションが行う唯一のネットワークリクエストです。個人データは一切送信されず、最新のバージョン番号を読み取るために、公開されているGitHubのリリースページへ安全な（HTTPS）リクエストを1つ送るだけです。XIMODが何かを自動的にダウンロードしたりインストールしたりすることは決してありません。ダウンロードリンクをたどるかどうかは常にあなたが選べます。

5. FOMODタグリファレンス

この節では、XIMOD Architectが実際に生成するタグを説明します。2つのファイルは、MODのfomodサブフォルダにUTF-8（バイトオーダーマーク付き）で書き込まれます。info.xmlは4スペース刻み、ModuleConfig.xmlはタブでインデントされます。読み込み時には、バイトオーダーマークによってエンコーディングが自動的に検出されるため、他のツールで生成されたUTF-8ファイルとUTF-16（LEまたはBE）ファイルの両方を開けます。

5.1 info.xmlファイル

<fomod>ルートの下のメタデータです。要素は下の表の順序で書き込まれます。「空でない場合」と記されたものは、対応するフィールドが空のままだと省略されます。

タグ	内容	書き込み
<Name>	MOD名。	常に
<Author>	作者。	空でない場合
<Version>	バージョン。	空でない場合
<Website>	MODのページのURL。	空でない場合
<Description>	説明。	空でない場合
<Groups>	MODのカテゴリ。	常に
<Game>	選択したゲームの識別子。XIMOD固有の拡張（FOMOD仕様外）で、MODマネージャーには無視されます。	空でない場合

完全な例：

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>My Awesome Mod</Name>
    <Author>Patrice</Author>
    <Version>1.0.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/12345</Website>
    <Description>A sample mod for the manual.</Description>
    <Groups>Player homes</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

5.2 ModuleConfig.xmlファイル

<config>ルートの下のインストールロジックで、ModConfig5.0.xsdスキーマを参照します。ブロックは次の順序で現れます。モジュール名、ヘッダー画像、必須ファイル、インストールステップ、条件付きインストール。

ルートとヘッダー

タグ／属性	役割
<config>	ルート。xmlns:xsiおよびxsi:noNamespaceSchemaLocation属性（ModConfig5.0.xsdスキーマ）を持ちます。

タグ／属性	役割
`<moduleName>`	モジュール名。ウィザードの上部に表示されます。
`<moduleImage path="…"/>`	ヘッダー画像（空のタグ。画像が定義されている場合のみ書き込まれます）。

ファイルとフォルダ

タグ／属性	役割
`<file source="…" destination="…" priority="…"/>`	ファイルのコピー。source = アーカイブ内のパス（ルートからの相対）、destination = Data以下の対象、priority = インストール順。
`<folder source="…" destination="…" priority="…"/>`	フォルダ全体のコピー（同じ属性）。
`<requiredInstallFiles>`	常にインストールされるファイルのコンテナ。
`<files>`	オプションまたは条件付きパターンに結び付いたファイルのコンテナ。

ステップとグループ

タグ／属性	役割
`<installSteps order="Explicit">`	ステップのコンテナ（定義された順序が保持されます）。
`<installStep name="…">`	ウィザードのページ。
`<visible>`	ステップの表示条件（<dependencies>を含みます）。
`<optionalFileGroups>`	ステップのオプショングループのコンテナ。
`<group name="…" type="…">`	グループ。type = 選択ルール（下記参照）。
`<plugins order="Explicit">`	グループのオプションのコンテナ。

<group>のtype属性の値：

type	制約
正確に1つ（必須） — SelectExactlyOne	ちょうど1つのオプション。
多くても1つ — SelectAtMostOne	多くても1つのオプション。
任意の数 — SelectAny	任意の数。
すべて（選択なし） — SelectAll	すべて、強制。
少なくとも1つ — SelectAtLeastOne	少なくとも1つ。

オプション（plugin）

タグ／属性	役割
`<plugin name="…">`	ユーザーに提供されるオプション。

タグ／属性	役割
`<description>`	オプションの説明テキスト（常に書き込まれます）。
`<image path="…"/>`	オプションを示す画像（定義されている場合）。
`<conditionFlags>`	オプションが選択されたときに設定されるフラグのコンテナ。
`<flag name="…">value</flag>`	フラグ。名前は属性に、値は内容に記述されます。
`<typeDescriptor>`	オプションのタイプ（単純または条件付き）を記述します。
`<type name="…"/>`	単純なタイプ（依存関係パターンが定義されていない場合）。
`<dependencyType>`	動的なタイプ（パターンが存在する場合）。
`<defaultType name="…"/>`	どのパターンも適用されない場合に適用されるデフォルトタイプ。

<type>／<defaultType>の値：

タイプ	ウィザードでの効果
オプション（任意） — Optional	未チェックのオプション。自由に有効化できます。
必須 — Required	必須のオプション。選択を解除できません。
推奨 — Recommended	デフォルトでチェックされるオプション。選択を解除できます。
使用不可 — NotUsable	無効なオプション（選択できません）。
使用できる可能性あり — CouldBeUsable	使用可能だが、潜在的にリスクがあるとして示されるオプション。

依存関係と条件

タグ／属性	役割
`<dependencies operator="…">`	条件のグループ。operator = AndまたはOr。
`<fileDependency file="…" state="…"/>`	ファイルに関する条件。state = Active、Inactive、またはMissing。
`<flagDependency flag="…" value="…"/>`	フラグとその期待値に関する条件。
`<conditionalFileInstalls>`	条件付きインストールのセクション。
`<patterns> / <pattern>`	パターンの一覧。各パターンは<dependencies>を<files>と（または<typeDescriptor>内では<type>と）関連付けます。

6. ModuleConfig.xmlファイルの詳細解説

第5章ではタグをリファレンス表として紹介しました。この章では、それらを1つずつ平易な言葉で順に説明し、各タグが何をするのか、どのようなオプションを受け付けるのか、そして何より、それらのオプションがMODのユーザーが目にするインストールウィザードにどのような具体的な効果を及ぼすのかを解説します。XMLの予備知識は不要です。XIMOD Architectがこのファイルをあなたの代わりに書きます。これを理解することは、より信頼性の高いインストーラーを設計するのに役立つだけです。

いくつかの用語について。XMLファイルは、山かっこで囲まれた「タグ」で構成されます。開始タグ<step>は終了タグ</step>と対になり、その2つの間に書かれたものがそのタグに属します。タグは「属性」、すなわち名前="値"の形式で書かれた設定を持てます。したがって、<group name="Resolution" type="SelectExactlyOne">は、2つの属性を持つ「group」タグです。最後に、内容のないタグは<file … />のように自己完結で閉じます（末尾の/に注意してください）。

6.1 インストーラーがファイルを読む仕組み

ユーザーがあなたのMODをインストールすると、そのマネージャー（Vortex、MO2、NMM）はModuleConfig.xmlを上から下へ読み、ウィザードを構築します。一連のページ（ステップ）で、それぞれがチェックするオプションのグループを提供します。最後に、マネージャーは選択されたオプションに一致するすべてのファイルを集め、ゲームにコピーします。心に留めておくべき3つの考え方があります。

- 順序が重要：ステップはファイル内に現れる順序で表示されます。
- チェックしてもすぐにはインストールされない：オプションをチェックしても選択が記録されるだけです。ファイルの実際のコピーはいちばん最後に行われます。
- 条件がすべてを調整できる：ステップを隠したり、オプションの状態を変えたり、追加ファイルのインストールをトリガーしたりします。これがフラグと依存関係の役割です（§6.6）。

6.2 ルートとヘッダー

<config> — ルート。設定全体が1つの<config>…</config>タグに収まります。これは、検証スキーマ（ModConfig 5.0）を指定する2つの技術的な属性（xmlns:xsiとxsi:noNamespaceSchemaLocation）を持ちます。これらに手を加える必要は一切ありません。XIMODは常に同一に書き込みます。

<moduleName> — MOD名。ウィザードの上部に表示されます。Mod情報タブで入力したものです。

<moduleImage path="…"/> — ウィザードの上部に表示されるバナー画像。path属性は、MODのルートからの相対で、その場所を示します（例：fomod\header.png）。このタグは空です（自己完結で閉じます）。画像がない場合は、単に省略されます。

6.3 ファイルのインストール

ゲームにコピーされるものを記述するタグが2つあります。<file>は単一のファイルをコピーし、<folder>はフォルダ全体をその内容すべてとともにコピーします。どちらも同じ3つの属性を受け付けます。

属性	役割	例
source	あなたのMODのアーカイブ内での要素のパス。ルートからの相対です。	tex\2k
destination	要素が配置されるべき場所。ゲームのDataフォルダからの相対です。	textures
priority	衝突時のインストール順（下記参照）。	0

priority属性とその影響。これは、2つのオプションが同じ場所にファイルをインストールするときのコピー順を決める数値です。ルールは、低い優先度が先に、高い優先度が後にインストールされ、最後に書かれたものが勝つため、高い優先度が低い優先度を「上書き」します。例：ベースのオプションがpriority="0"でテクスチャをインストールし、「高解像度」のオプションが同じテクスチャをpriority="1"でインストールすると、オプションの表示順にかかわらず、残るのは高解像度版です。意図的に調整すべき衝突がない限り、どこも0のままにしておきましょう。

> 注 —
> destinationはデフォルトで空で、その場合は要素がゲームのDataフォルダのルートにインストールされます。要素を別の場所に置く必要があるときは、destinationセルをクリックして明示的なパス（Dataからの相対）を入力します。ソースフォルダがすでにゲームの構造を反映している場合は、空のdestinationが正しい選択です。

これらのタグは3か所に現れます。<requiredInstallFiles>内（常にインストールされるファイル）、オプションの<files>内（オプションが選択された場合にインストール）、条件付きパターンの<files>内（条件が満たされた場合にインストール）です。

6.4 ステップ、グループ、オプション

これはウィザードの中核です。階層は単純で、ステップはグループを含み、グループはオプションを含みます。

<installSteps order="Explicit">はすべてのステップをまとめます。order="Explicit"は「私が書いた順序を尊重する」という意味です。ウィザードの各ページは<installStep name="...">で、そのname属性はページ上部に表示されるタイトルです。

<visible> — ステップを条件付きで表示します。ステップは、特定の条件が真の場合にのみ表示できます。その条件を<visible>タグに置きます（§6.6を参照）。<visible>がなければ、ステップは常に表示されます。例：ユーザーが先に「エキスパートインストール」をチェックした場合にのみ「詳細設定」ステップを表示します。

<optionalFileGroups>はステップのグループを含みます。各<group name="..." type="...">は、名前（セクションのタイトル）と、何より、選択ルール、すなわちユーザーが何をチェックできるかを決めるtypeを持ちます。

type	ユーザーに見えるものと、できること
正確に1つ（必須） — SelectExactlyOne	ラジオボタン：ちょうど1つのオプションを選ばなければなりません。0でも2でもいけません。排他的なバリアント（単一のテクスチャ解像度）に最適です。
多くても1つ — SelectAtMostOne	上と同じですが、何も選ばないこともできます。「オプションまたは何もなし」です。
任意の数 — SelectAny	独立したチェックボックス：好きなだけ、なしからすべてまでチェックします。積み重ね可能なモジュールに最も一般的なケースです。
少なくとも1つ — SelectAtLeastOne	チェックボックスですが、続けるには少なくとも1つチェックしなければなりません。手ぶらで先に進むのを防ぎます。
すべて（選択なし） — SelectAll	すべてのオプションが自動的にチェックされてロックされます。ユーザーは選択を解除できません。必須ファイルを説明しつつ提示するのに使います。

グループ内では、<plugins order="Explicit">がオプションを含み、それぞれが<plugin name="…">です（ここでの「plugin」という語は、.espファイルではなくウィザードのオプションを意味します）。そのnameは、ユーザーがチェックするラベルです。オプションは次のもので構成されます。

- <description>：オプションが選択されたときに右側に表示されるテキスト。そこにその効果を明確に記述します。
- <image path="…"/>：説明とともに表示される図（任意）。
- <conditionFlags>：オプションが選択されたときに設定されるフラグ（§ 6.6）。
- <files>：オプションが選択された場合にインストールされるファイル。
- <typeDescriptor>：オプションのデフォルトの状態（§ 6.5）。

6.5 オプションのタイプとその挙動

各オプションは、ウィザードでのデフォルトの状態（チェックされるかどうか、編集できるかどうか）を決める「タイプ」を持ちます。これは<typeDescriptor>で記述されます。単純なケースでは、単一の<type name="…"/>を含みます。5つの値とその効果は次のとおりです。

type	ウィザードでの効果
オプション（任意） — Optional	デフォルトで未チェック。自由にチェック・チェック解除できます。一般的なケースです。
推奨 — Recommended	デフォルトでチェック済みですが、ユーザーはチェックを解除できます。推奨する選択に使います。
必須 — Required	チェック済みでロック：チェックを解除できません。不可欠なコンポーネントに使います。
使用不可 — NotUsable	未チェックでロック：チェックできません。現在の状況では利用できないオプションであることを示します。
使用できる可能性あり — CouldBeUsable	チェック可能ですが、潜在的に問題があるとして示されます（多くは警告付き）。

動的なタイプ：<dependencyType>。オプションのタイプは、条件に応じて変化させることもできます。固定の<type>の代わりに、<defaultType name="…"/>（どの条件も満たされないときのデフォルトタイプ）と<pattern>の一覧を含む<dependencyType>を書きます。各patternは、条件を、それらが真の場合に適用する<type>と関連付けます。

例：「ENB Ultraプリセット」オプションは、デフォルトではOptionalで、ユーザーが動的な蝋燭を有効にするとRecommendedになり、必須ファイルがディスク上に欠けているとNotUsableになる、というようにできます。インストーラーを「スマート」にするのは、この仕組みです。完全な例は付録Dに掲載しています。7.5節では、この仕組みを使って「プリセット」を、ユーザーが後から調整できるデフォルトに変えています。

6.6 フラグ、依存関係、条件付きインストール

この3つの概念がFOMODの条件システムを形成します。インストーラーがユーザーの選択に反応できるのは、これによります。

フラグ — <conditionFlags>と<flag>。フラグは、オプションが選択されたときに設定する小さな変数です。オプションの<conditionFlags>タグは、1つ以上の<flag name="…">値</flag>を含みます。nameはフラグ名で、タグの間のテキストがその値です。したがって、<flag name="res">4K</flag>は、オプションがチェックされると同時に「res」フラグを「4K」に設定します。フラグそれ自体には目に見える効果はありません。メモリとして働き、後で別の場所で条件として再利用されます。

依存関係 — <dependencies operator="…">。依存関係は、確認すべき条件です。これらは<dependencies>にまとめられ、そのoperator属性が組み合わせ方を指定します。And（すべての条件が真でなければならない）またはOr（1つで十分）です。条件には2種類あります。

条件	テストする内容
`<flagDependency flag="…" value="…"/>`	指定された値を名前付きフラグがちょうど持っている場合に真。これが、先に行われた選択に反応する方法です（例：「res」フラグが「4K」に等しい）。
`<fileDependency file="…" state="…"/>`	ユーザーのゲームに既に存在する、または存在しないファイルの状態に応じて真。

ファイル依存関係のstate属性は3つの値を取ります。

state	意味
Active	ファイルがゲームに存在し、有効になっている。
Inactive	ファイルは存在するが、無効になっている。
Missing	ファイルが存在しない。

これらの依存関係は3つのことに使われます。ステップの表示可否（<visible>内）、オプションの動的なタイプ（<dependencyType>の<pattern>内）、そして下記の条件付きインストールです。

条件付きインストール — <conditionalFileInstalls>。ステップの後に置かれるこのセクションは、ユーザーが直接何かをチェックしなくても、条件が満たされたときにのみファイルをインストールします。<pattern>要素を含み、各要素は<dependencies>を<files>と関連付けます。パターンの条件がウィザードの終了時に真であれば、そのファイルがインストールされます。

ステップの表示可否との違い：<visible>はページを表示するかどうかを決めるのに対し、<conditionalFileInstalls>は裏側でファイルをインストールするかどうかを決めます。典型的なケース：ユーザーが特定の2つのオプション（それぞれフラグが設定される）をチェックした場合にのみ、互換性パッチを自動的にインストールします。

> 注 —
> フラグは、依存関係によってテストされる前に、まずオプションによって（<conditionFlags>経由で）設定されていなければなりません。したがって、ステップでフラグを定義してから、条件付きパターンでそれらをテストしてください。
>
> 注 — 設計ルール：オプションが1つの決定 —
> プリセット、プロファイル、「モジュール」のオン／オフ —
> を表すだけなら、そのオプションにはフラグを設定させるだけにして、コピーはそれらのフラグをキーとする条件付きインストールに任せます。そうすれば、機能ごとに1つのフラグが唯一の信頼できる情報源になります。選択がどう組み合わされても、プリセットと手動で選んだオプションが同じファイルを二重にインストールすることは決してありません。7.5節はこのルールを適用しています。

7. ModuleConfig.xmlの例

以下の例は、最も一般的なケースを示します。これらは、XIMOD Architectが実際に生成するXMLに対応しています（読みやすさのためにインデントは簡略化しています）。

7.1 最小の例 — 必須ファイル

選択のないMOD：すべてのファイルが直接インストールされます。

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>My Simple Mod</moduleName>
    <requiredInstallFiles>
        <file source="MyMod.esp" destination="MyMod.esp" priority="0"/>
        <folder source="textures" destination="textures" priority="0"/>
    </requiredInstallFiles>
</config>
```

7.2 排他的な選択 — SelectExactlyOne

単一のテクスチャ解像度の選択を提供するステップ。2Kオプションがデフォルトで推奨されます。

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>HD Textures</moduleName>
    <moduleImage path="fomod\header.png"/>
    <installSteps order="Explicit">
        <installStep name="Texture resolution">
            <optionalFileGroups>
                <group name="Resolution" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="2K">
                            <description>2K textures (balanced).</description>
                            <files>
                                <folder source="2k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Recommended"/>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="4K">
                            <description>4K textures (high quality).</description>
                            <files>
                                <folder source="4k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
</config>
```

7.3 条件付きインストール — フラグ

あるオプションがチェックされるとフラグを設定し、そのフラグが有効な場合にのみファイルがインストールされます。

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Conditional patch</moduleName>
    <installSteps order="Explicit">
        <installStep name="Options">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enables module A.</description>
                            <conditionFlags>
                                <flag name="ModuleA">On</flag>
                            </conditionFlags>
                            <files>
                                <folder source="A" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
    <conditionalFileInstalls>
        <patterns>
            <pattern>
                <dependencies operator="And">
                    <flagDependency flag="ModuleA" value="On"/>
                </dependencies>
                <files>
                    <file source="patchA.esp" destination="patchA.esp" priority="0"/>
                </files>
            </pattern>
        </patterns>
    </conditionalFileInstalls>
</config>
```

7.4 1ページ、複数ステップ — SelectAny + フラグ

1つの複数選択ステップは、プレイヤーがチェックした各オプションごとに異なるステップを開けます。ステップはオプションに結び付いていません。各オプションが条件フラグを設定し、各対象ステップがそのフラグに関する表示条件を持ちます。AとBをチェックするとステップAとBが表示され、A、B、Cをチェックすると3つのステップが表示されます。

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Modular Add-ons</moduleName>
    <installSteps order="Explicit">
        <!-- Step 1: the choices. Each ticked option sets its own flag. -->
        <installStep name="Choose modules">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enable module A.</description>
                            <conditionFlags>
                                <flag name="want_A">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="Module B">
                            <description>Enable module B.</description>
                            <conditionFlags>
                                <flag name="want_B">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- Modules C and D follow the same pattern (want_C, want_D). -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step A: shown only when want_A is on. -->
        <installStep name="Module A options">
            <visible>
                <flagDependency flag="want_A" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module A" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="A - extra textures">
                            <description>Optional high-res textures for A.</description>
                            <files>
                                <folder source="a_textures" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step B: shown only when want_B is on. -->
        <installStep name="Module B options">
            <visible>
                <flagDependency flag="want_B" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module B" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="B - extra meshes">
                            <description>Optional meshes for B.</description>
                            <files>
                                <folder source="b_meshes" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Steps for C and D have exactly the same shape, each guarded by its
             own <visible><flagDependency flag="want_C | want_D" value="on"/>. -->
    </installSteps></config>
```

7.5 カスタマイズ可能なデフォルト付きプリセット

「プリセット」とは、複数の選択を一度に代行するオプションです。ユーザーはその両方を求めます。プリセットを選んでそれで終わりにすることと、プリセットを選んだうえで1つか2つの機能を調整することです。鍵は、決定とインストールを分けることです。プリセットは決定します — フラグを設定します。機能ページでは、ユーザーがその決定を見直せます — 同じフラグをもう一度設定します。コピーを行うのは条件付きインストールで、各フラグの最終値に基づきます。

ステップ1ではプリセットを提示し、その横に「カスタマイズ」グループを置きます。このグループのオプションは、ユーザーが確認したい機能のページを開くだけです。各機能ページは、対応する「カスタマイズ」オプションがチェックされているときに表示されます（§ 7.4）。そのページのオプションは動的なタイプ（§ 6.5）を使います。プリセットのフラグが設定されている場合は、プリセットがインストールするはずだったバリアントがRecommended — 事前にチェック済み — になり、そうでない場合は「なし」が推奨されます。ページはプリセットの後に来るので、ページが機能フラグに書き込む値がプリセットの値を置き換えます。ユーザーはプリセットのバリアントをそのまま使うことも、別のバリアントに切り替えることも、機能を外すこともできます。

プリセットにもページのオプションにも`<files>`は一切ありません。コピーはすべて`<conditionalFileInstalls>`で、フラグ値ごとに1つのパターンとして行われます。これが二重インストールを不可能にする仕組みです。選択がどう組み合わされても、各機能フラグの最終的な値はちょうど1つ、したがってファイルのセットも1つになります。

> 注 —
> MODマネージャーはフラグをページごとに順に評価するため、後のページで行われた設定が、プリセットによる設定を置き換えます。プリセットは最初のページに置き、すべての機能ページにフラグを明示的にクリアする「なし」オプションを用意してください。そうしないと、すべてのチェックを外しただけのユーザーには、プリセットの値が残ってしまいます。

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>カスタマイズ可能なデフォルト付きプリセット</moduleName>
    <installSteps order="Explicit">
        <!-- ステップ1：プリセットは、含まれるすべての機能のフラグを設定するだけで、ほかには何もしません。 -->
        <installStep name="インストールモード">
            <optionalFileGroups>
                <group name="プリセット" type="SelectAtMostOne">
                    <plugins order="Explicit">
                        <plugin name="プリセットA（推奨構成）">

<description>機能Aをバージョン2で、機能Bをバージョン1でインストールします。</description>
                            <conditionFlags>
                                <flag name="preset">on</flag>
                                <flag name="feat_A">v2</flag>
                                <flag name="feat_B">v1</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
                <group name="カスタマイズ" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="機能Aをカスタマイズ">
                            <description>機能Aのページを開きます。</description>
                            <conditionFlags><flag name="cust_A">on</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
```

```
                    </plugin>
                    <!-- 「機能Bをカスタマイズ」、「機能Cをカスタマイズ」...はcust_B、cust_Cを設定します。
-->
                </plugins>
            </group>
        </optionalFileGroups>
    </installStep>
    <!-- ステップ2：機能A。ユーザーがカスタマイズを求めたときだけ表示されます。 -->
    <installStep name="機能A">
        <visible><flagDependency flag="cust_A" value="on"/></visible>
        <optionalFileGroups>
            <group name="機能A" type="SelectExactlyOne">
                <plugins order="Explicit">
                    <plugin name="なし">
                        <description>機能Aをインストールしません。</description>
                        <conditionFlags><flag name="feat_A">none</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Recommended"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="on"/>
                                        </dependencies>
                                        <type name="Optional"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                    <plugin name="バージョン1">
                        <description>機能A、バージョン1。</description>
                        <conditionFlags><flag name="feat_A">v1</flag></conditionFlags>
                        <typeDescriptor><type name="Optional"/></typeDescriptor>
                    </plugin>
                    <plugin name="バージョン2">
                        <description>機能A、バージョン2（プリセットの選択）。</description>
                        <conditionFlags><flag name="feat_A">v2</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Optional"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="on"/>
                                        </dependencies>
                                        <type name="Recommended"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                </plugins>
            </group>
        </optionalFileGroups>
    </installStep>
    <!-- 機能Bと機能Cのページもまったく同じ形です。 -->
</installSteps>
<!-- ファイルのコピーはここでのみ、各機能フラグの最終値に基づいて行われます。 -->
<conditionalFileInstalls>
    <patterns>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_A" value="v1"/></dependencies>
            <files><folder source="FeatureA\v1" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_A" value="v2"/></dependencies>
            <files><folder source="FeatureA\v2" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_B" value="v1"/></dependencies>
            <files><folder source="FeatureB\v1" destination="" priority="0"/></files>
        </pattern>
    </patterns>
</conditionalFileInstalls>
```

```
</config>
```

プロジェクト → インストーラのプレビュー…では、各組み合わせ — プリセットのみ、プリセット＋「機能Aをカスタマイズ」、「機能Aをカスタマイズ」のみ — を順にたどり、公開前に最終的なファイル一覧を確認できます（§ 4.11、§ 8.19）。プロジェクト → FOMODを検証は、宛先の衝突を報告し、このプリセット／カスタマイズのパターンを認識します（§ 8.1）。

7.6 導入ページと終了ページ

ウィザードを描画するのはMODマネージャーであり、インストーラーではありません。作者が制御できるのは、ステップ、グループ、オプションの名前、説明、画像だけで、それ以外は何も制御できません。フォントも、色も、レイアウトも、自由なテキストブロックもありません。その範囲内であれば、ようこそページと終了ページは簡単に作れます。

ようこそページ —

最初のステップを、Requiredオプションを1つだけ含むSelectAllタイプの単一グループにします。その説明には歓迎のテキストを書きます。マネージャーはこれを説明エリアに表示します。その画像にはビジュアル — バナー、ロゴ、あるいは好きな書体で画像化したテキスト — を入れます。ファイルは何も割り当てないか、そこに一覧表示したい場合はMODの基本ファイルを割り当てます。

終了ページ —

最後のステップに同じ構成を使います。お別れのメッセージと、必要に応じて次にすべきこと（ロードオーダー、インストールすべきパッチ）を記したRequiredオプションです。インストーラーはユーザーの選択内容の概要を表示できません。この形式では、あるページから他のページで行われた選択にアクセスできないからです。マネージャーはこれからインストールされるファイルの一覧を独自に表示し、インストーラーのプレビュー（§ 4.11）では、作者であるあなたがあらゆるシナリオについてその概要を確認できます。

```
<installStep name="ようこそ">
    <optionalFileGroups>
        <group name="このインストーラーについて" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="Dawnlightへようこそ">

<description>Dawnlightをダウンロードしていただき、ありがとうございます。次のページでは、ライティングのプリセット、オプションのモジュール、互換性パッチを選択できます。最後のページまで何もインストールされません。</description>
                    <image path="fomod\welcome.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
<!-- ... 実際のステップ ... -->
<installStep name="完了">
    <optionalFileGroups>
        <group name="プレイする前に" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="ありがとうございました！">

<description>Dawnlight.espは天候MODの後に配置してください。問題があればNexusのページでご報告ください。どうぞお楽しみください！</description>
                    <image path="fomod\thanks.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
```

```
    </optionalFileGroups>
</installStep>
```

ようこそステップをテンプレートとして保存しておけば（プロジェクト → テンプレート…、§ 8.3）、どのMODでも再利用できます。

8. バージョン2 — 新機能

以下の機能はバージョン2.0.0で導入されました。追加された順に紹介します。第2章から第7章までの番号付き節には、すでにこれらが反映されています。8.26節から8.29節は、バージョン2.1.0で追加されました。8.30節から8.32節は、バージョン2.2.0で追加されました。

8.1 宛先衝突の検出

プロジェクトを検証する（プロジェクト → FOMODを検証）と、XIMODは、互いを上書きしてしまうような、異なるファイルを同じ宛先パスにインストールするオプションも報告するようになりました。同時に発生しうる衝突は「確実」として、相互排他的なオプションにまたがってのみ発生する衝突は「可能性あり」として示されます。

衝突が「確実」として報告されるのは、2つのオプションが本当に同時に選択できる場合のみです。排他的なグループ（SelectExactlyOne／SelectAtMostOne）に加えて、XIMODはフラグも追跡するようになりました。あるオプションがフラグをある値に設定し、もう一方のオプションが、そのフラグの別の値のときにのみ表示されるページ上にあり、かつ最初のオプションと並んで選択できるものがその別の値を設定できない場合、そのペアは共存できず、衝突は潜在的なものとしてのみ列挙されます。典型的なケース：「プリセット」オプションと、「自分で選ぶ」ページの対応するオプションです。

8.2 7zエクスポート

エクスポートコマンドは、ZIPに加えて7-Zip（.7z）も提供するようになりました。保存ダイアログで形式を選びます。通常、7zはより小さいダウンロードを生成します。

8.3 再利用可能なテンプレート

プロジェクト → テンプレート…では、選択中のステップを再利用可能なテンプレートとして保存したり、保存済みのテンプレート（ステップ、グループ、またはオプション）を任意のプロジェクトに挿入したりできます。テンプレートはアプリケーションの設定フォルダに保存されます。

8.4 フォルダから新規

ファイル → フォルダから新規…は、フォルダをスキャンし、直下の各サブフォルダが1つのオプションになるスケルトンを提案します。ルートにある明らかなヘッダー画像は自動的に検出されます。

8.5 ドラッグ＆ドロップでのファイル割り当て

ツリーでオプションを選択した状態で、ファイルエクスプローラー（Windowsのエクスプローラー、Finder、Linuxのファイルマネージャー）からファイルやフォルダをXIMODのウィンドウへドラッグすると、「ファイルの追加」／「フォルダの追加」とまったく同じように、そのオプションのソース表に追加されます。複数の項目を一度にドロップすることもできます。受け付けられるのはプロジェクトのルートフォルダ以下のパスだけで、それ以外は無視されます。大規模なインストーラーでは、これがソースを割り当てる最速の方法です。エクスプローラーで機能フォルダを一度開き、各サブフォルダを対応するオプションにドロップしてください。

8.6 FOMODの比較

プロジェクト →
比較…は、現在のプロジェクトを別のFOMODと比較し、追加、削除、変更された点（ステップ、オプション、ファイル、メタデータ）を一覧表示します。変更履歴を書くのに便利です。

8.7 ビジュアル条件エディター

プロジェクト →
条件エディター…は、各条件フラグについて、それを設定するオプションと、それを使用するステップの表示ルール、条件付きインストール、依存関係パターンを表示します。設定されているが一度も使われていない（または使われているが一度も設定されていない）フラグは強調表示されます。

8.8 画像のチェックと最適化

検証は、大きすぎる、未対応の形式である、または読み取れないヘッダー／オプション画像について警告するようになりました。Mod情報タブでは、画像を最適化がヘッダー画像を適切な範囲まで縮小します。

8.9 コマンドラインの追加

コマンドラインに次が追加されました。inspect
<plugin.esp>（プラグインのマスター、作者、ESLフラグを表示）、batch
<manifest.json>（継続的インテグレーション向けに、複数のFOMODを検証・ビルド・パッケージ化）、そしてba2 <dir> <out.ba2>
[game]（フォルダを一般BA2アーカイブにまとめる）です。BA2のパッケージ化は実験的です。出荷前にゲーム内でアーカイブを確認してください。

8.10 バックグラウンドでのエクスポートと検証

配布アーカイブのエクスポートでウィンドウが固まらなくなりました。FOMODの書き込み後、圧縮はバックグラウンドで実行され、進捗ウィンドウが処理済みのファイル数と追加中のファイルを表示します。キャンセルボタンでエクスポートを中止し、途中のアーカイブを削除します。同様に、検証のディスクチェック（欠落ファイル、宛先の衝突、画像）もバックグラウンドで実行されます。レポートウィンドウは完了まで「ディスク上のファイルを確認中…」と表示し、その間にプロジェクトが変更された場合は警告します。

8.11 クラッシュからの復旧

XIMOD
Architectが予期せず終了した場合、未保存の変更があるすべてのプロジェクトが、アプリケーションの設定ディレクトリ以下の復旧フォルダ（crash_recovery/<日付>/<MOD名>/fomod/）に書き込まれます。次回起動時、ステータスバーにバックアップの保存場所が表示されます。そのフォルダをファイル → フォルダを開く…で開いて作業を取り戻してください。

8.12 より分かりやすいインターフェース

日常的な不便がいくつか解消されました。まだフォルダのないプロジェクトを保存すると、何もしないのではなく保存先を尋ねるようになり、ファイル →
名前を付けて保存…（Ctrl+Shift+S）でプロジェクトを別のフォルダへ移せます。ウィンドウのタイトルには、未保存の変更がある間、MOD名と●ドットが表示されます。成功、警告、エラーは右上隅に通知として表示されます（エラーは閉じるまで残ります）。一方、ステータスバーは最後のメッセージとステップ・オプションの数を保持します（ルートフォルダはインスペ

クターのワークスペースセクションに、MOD名はウィンドウのタイトルとドキュメントタブに表示されます）。ルートフォルダ外のファイルを選択すると、無言で無視されるのではなく報告されるようになりました。
用語 —
インターフェースは、FOMOD形式でpluginと呼ばれるもの（グループで提供される選択肢）を、すべての言語で一貫して「オプション」と呼ぶようになりました。「プラグイン」はゲームのプラグインファイル（.esp、.esm、.esl）に対して使われます。テーマ設定の「システム」は、オペレーティングシステムにリアルタイムで追従します。OSをライトとダークモードの間で切り替えると、XIMODも再起動なしで即座に切り替わり、選んだフォントサイズは両方で保たれます。最後に、データセット（ゲーム、国、言語）は、スプラッシュスクリーンの表示後ではなく表示中に読み込まれるようになり、メインウィンドウがより早く現れます。

8.13 既存のFOMODの翻訳

プロジェクト →
FOMODを翻訳…は、インストーラーのユーザー向け文字列を別の言語に翻訳する専用ウィンドウを開きます。MOD名と説明、ステップ・グループ・オプションの名前、オプションの説明です。フラグ、ファイルパス、タイプには決して手を加えません。MODマネージャーがそれらを文字どおりに比較するためです。アクティブなプロジェクト（保存済みである必要があります）を読み込むか、MODフォルダを開く…で任意のMODフォルダを読み込みます。ソースとターゲットの言語を選び、翻訳列を埋めます。名前は表で直接、説明は下部のパネルで（ソースが横に並んで表示され、スペースや改行は任意で可視化できます）。フィルター（未翻訳、要確認、問題あり、ロック済み）、検索ボックス、進捗バー、Ctrl+Enter（次の未翻訳へ）が進捗の把握に役立ちます。作者とウェブサイトはデフォルトでロックされており、ロックされた文字列は変更されずに書き込まれます。
問題は各行に示されます。翻訳に欠けている保護されたトークン（ファイル名、URL、リテラルの\nシーケンス）、名前の中の改行、XMLが保存できない文字、ソースと同一の翻訳、または同じソーステキストが他の場所で異なって翻訳されていること、です。ブロッキングの問題はエクスポートを妨げます。作業は、サイドカーファイルfomod/translations/<mod>.<言語>.ximod-
translationとして保存され（Ctrl+S）、次にそのフォルダを開いたときにウィンドウが自動的に再読み込みします。フォルダから更新はFOMODを読み直し、新規・変更・移動・削除された文字列を報告し、既存の翻訳を保持します（変更されたソースは「要確認」とマークされます）。
翻訳済みファイルをエクスポートは、info.xmlとModuleConfig.xmlを、隣接するフォルダfomod_<言語>（デフォルト、元のファイルはそのまま）に書き込むか、タイムスタンプ付きの.bakコピーを取った後に元のファイルへ上書きします。エクスポートはロスレスです。変更されるのは翻訳された文字列のみで、元のXMLのコメント、エンコーディング（UTF-8またはUTF-16）、未知の要素、書式はバイト単位で保持されます。「元の順序を保つ」はorder="Explicit"を強制し、名前でソートされた一覧が翻訳後の名前で並べ替えられないようにします。翻訳をプレビューは、翻訳されたコピーでインストーラーのシミュレーションを開きます。同じ操作はコマンドラインにもあります。ximod-architect translate extract | status | update | apply（§5を参照）。
翻訳パッケージ —
3つ目のエクスポートモードは、Nexusにアップロードできるアーカイブを構築します。デフォルトは「パッチのみ」のパッケージで、翻訳されたinfo.xmlとModuleConfig.xml（元のパスとエンコーディングのまま）とターゲット言語のREADMEだけを含み、ユーザーはこれを元のMODに上書きインストールします。「フルMOD」オプションは代わりにMODのすべてのファイルを含みます（作者が再配布を許可しているか確認してください）。形式（.zipまたは.7z）とファイル名は名前テンプレートに従い、トークン{name}、{version}、{LANG}、{lang}、{lang3}、{langn

ame}が使えます（デフォルトは「{name}_{LANG}」、例：MyMod_FR.zip）。アーカイブは、いつもの進捗ウィンドウとともにバックグラウンドで構築されます。
生産性 —
言語ペアごとの翻訳メモリが設定フォルダに保存され、保存したすべての翻訳を学習します。「メモリから補完」は、次のMODで同一の文字列をあらかじめ埋め（「要確認」とマークされます）、下部パネルは選択した文字列について記憶された翻訳を提案します。「同一のものへ反映」は、現在の翻訳を、同じソーステキストを持つ他のすべての文字列へコピーします。用語集（ツール行で切り替え）は、指定された用語 —
用語とその期待される翻訳（任意で大文字小文字を区別）、または翻訳せずに保つ用語 —
を列挙し、ルールに反する行は問題として示されます。CSVエクスポート／CSVインポートは、表全体（キー、コンテキスト、ソース、翻訳、ステータス、メモ）をスプレッドシートや別の翻訳者とやり取りします。インポートは行をキーで照合し、Excelの区切り文字も許容します。
ユニークなテキスト —
フィルター行の切り替えは、個別のソーステキストごとに1行を表示し、グループのサイズを示す×Nバッジを付けます。その行を翻訳すると、同じテキストを持つすべての文字列が、確認なしで自動的に一度に翻訳されます（ロックされた文字列を除く）。そして、いくつ更新されたかをメッセージが示します。多くのページで同じオプション名や説明が繰り返されるFOMODを最速で処理する方法です。オフの場合、表はすべての文字列を表示し、「同一のものへ反映」は、未翻訳のものだけへの一回限りのコピーとして引き続き利用できます。
キーボードと空白 — 表では、↑と↓で選択行を移動し、Page UpとPage Downで1画面分の行をジャンプ、HomeとEndで最初と最後の行へ移動します（翻訳フィールドに入力していない間。同じキーはプロジェクトの文字列ウィンドウでも動作します）。「空白と改行を表示」は、ソーステキストのすべての空白を、表と詳細ペインの両方で可視化します。スペースは·、改行なしスペースは␣、タブは→、復帰は␍、各行末には¶です。
コマンドラインでは：ximod-architect translate package <root> <sidecar> [-o OUT] [--format zip|7z] [--full]、translate export-csv / import-csv、translate tm-learn / tm-apply。また、batchのマニフェストエントリは「translations」の一覧（sidecar、mode、out、format、full）を持つことができ、メインアーカイブとともに翻訳パッケージを生成できます。

8.14 MODアーカイブを開く

ファイル →
アーカイブを開く…（または.zip/.7zファイルをウィンドウにドロップ）は、アーカイブを設定ディレクトリの作業フォルダ（work/<アーカイブ名>）にバックグラウンドで、いつもの進捗ウィンドウとともに展開し、その中にあるFOMODを開きます。fomodフォルダがアーカイブの最上位にあっても、単一の包むフォルダの下にあってもかまいません。すでに展開済みのアーカイブはそのまま再利用されるため、そこで行った編集は保たれます。.rarアーカイブには対応していません。先に7-Zipで展開してください。翻訳ウィンドウにも同じボタン、MODアーカイブを開く…があり、fomodフォルダのみを展開します（translate_work/<アーカイブ名>へ）。その場合、MODのファイルがそこにないため、「フルMOD」パッケージオプションは利用できません。
コマンドラインでは：ximod-architect extract <archive> [-o DIR] [--fomod-only]は、アーカイブを展開し（デフォルトではそれにちなんだ名前のフォルダへ）、FOMODのルートを表示します。

8.15 インポートの忠実性

XIMODは、以前は無言で削除していた構造を読み込んで書き戻せるようになりました。MOD全体の要件（moduleDependencies。Mod情報タブの「MODの要件」で、他と同じ依存関係エディターで編集します）、ファイルとフォルダのalwaysInstallおよびinstallIfUsable属性（ファイル表の「常に」と「使用可能な場合」の列）、タイトルの位置と色、ヘッダー画像のオプションsh

owImage・showFade・height（Mod情報 →
「詳細ヘッダー」）、そしてステップ・グループ・オプションのAscending/Descending順序属性（記述されたとおりに保持されます）です。
まだモデル化できないもの — XIMODが知らない、スキーマ外の要素 —
は、プロジェクトの読み込み時に報告されます。警告「このFOMODには、XIMODが編集できない構造がN個含まれています。プロジェクトの保存時に削除されます」が「問題」パネルを開き、各構造とその場所（ステップ、オプション、条件付きセット）を1行ずつ表示します。行をクリックするとノードが選択されます。これらの警告は、プロジェクトが開いている間、検証をまたいで「問題」パネルに残ります。ximod-architect validateも同じ警告を表示します。

8.16 バックアップと復旧

既存のバージョンに上書きしてプロジェクトを保存するたびに、まず以前のinfo.xmlとModuleConfig.xmlがfomod/backups/<日時>/にコピーされ（これから書き込むものと同一の場合は何もコピーされません）、最新のN個のフォルダのみが保持されます。Nは「保持するバックアップ数」の設定です。そのフォルダは配布アーカイブやBA2パッケージに含まれることは決してなく、そのファイルが孤立ファイルとして報告されることもありません。両方のXMLファイルはアトミックに書き込まれるため（一時ファイル、フラッシュ、リネーム）、保存中にクラッシュしても書きかけのインストーラーが残ることはありません。
ファイル →
バックアップを復元…は、バックアップを新しい順に、それぞれ現在のプロジェクトとの差異の数とともに一覧表示します。比較はそのバックアップに対していつもの比較ウィンドウを開き、復元は取り消し可能な単一の変更としてエディターに読み込み（保存するまでディスクには何も書き込まれません）、すべてのバックアップを削除は2回目のクリックで確定してフォルダを空にします。§
8.11のクラッシュ対策に加えて、変更済みの各プロジェクトの復旧コピーが一定間隔で書き込まれるようになりました（「N分ごとの復旧コピーの自動保存」）。これは正常終了時に消去されるため、復旧の確認は異常終了後にのみ表示されます。
コマンドラインでは：ximod-architect backups <root> --list、--restore <timestamp>、または--prune N。buildとpackageも書き込み前にバックアップを作成します。

8.17 マスターを自動条件に

プラグインファイル（.esp、.esm、.esl）がオプションに追加されると（ファイル表からでもドラッグ＆ドロップでも）、XIMODはそのヘッダーを読みます。プロジェクトにまだ作者がない場合は、プラグインの作者が入力されます。また、そのプラグインが必要とするマスターのうち、ゲーム自身のファイル（Categories.jsonの"masters"にゲームごとに列挙）でも、このMODがインストールするものでもないものは、すべてオプションのファイル条件「Active」となり、オプションのデフォルトタイプの最初の依存関係パターン（必要なら作成されます）に追加されます。オプションのファイル条件によってすでに指名されているマスターが二重に追加されることはありません。「プラグインのマスターを条件として追加する」設定でこれをオフにできます。
検証は、プロジェクトがインストールするすべてのプラグイン（必須ファイル、オプション、条件付きセット）のヘッダーを読み、MOD内にもゲームのベースファイルにも依存関係としての宣言にも含まれないマスターについて警告します。「XにはYが必要ですが、これはこのMOD内になく、依存関係としても宣言されていません」。この警告はノードを対象とするため、「問題」パネルでクリックするとそのオプションが選択されます。

8.18 ライトプラグイン（ESL）とプラグインレポート

検証は、ライトプラグインのルールも確認します。lightフラグが設定されていない.esl拡張子のファイル、新規レコードが上限を超えるか許可されたFormID範囲外にあるlightフラグ付きプラ

グイン、そして ― 情報として ―
lightフラグを付けられる可能性のある.esp（プラグインに新しいレコード、すなわちFormIDのインデックスがマスター数に等しいものを数え、ゲームの上限と比較します。デフォルトは2048、Starfieldは4096で、Categories.jsonの"eslLimit"でゲームごとに調整できます）です。64MBを超えるファイルは数えられません。CLIのinspect <plugin>も同じ数値を表示します。
ツール →
プラグインレポート…は、それらすべてを1つの表にまとめます。ファイル、種類、lightフラグ、マスター（欠落しているものは警告色で）、上限に対する新規レコード数、適格性です。読み取れないファイルはそのように表示されます。

8.19 より豊かなシミュレーターとシナリオ

インストーラープレビュー（プロジェクト →
プレビュー）の概要ページは、最後に最終ファイルツリーで終わるようになりました。ゲームのDataフォルダに配置されるすべてのファイル、ディスクから展開されサイズ付きで表示されるフォルダ、そしてインストールの合計サイズです。2つのエントリが同じ最終パスに配置される場合、優先度の高い方が勝ち（同点の場合は後の宣言が優先）、負けた方は取り消し線付きで表示され、ホバーすると勝ったオプションが表示されます。20,000エントリを超えるツリーは切り詰められ、その旨が表示されます。
ファイルツリーは、XIMODの他のすべてのツリーと同じルールに従います。フォルダを右クリックすると、展開と折りたたみのサブメニュー（すべて／選択項目／選択項目から。最後のものは、インストールされたアーカイブの内容を含め、フォルダとその配下すべてに作用します）が開きます。テキストフィールドにフォーカスがない間は、↑と↓でカーソルリングを行から行へ移動し、HomeとEndで端へジャンプ、→でフォルダを開き（開いているものの中に入り）、←で閉じ（ファイルからそのフォルダへ戻り）、Enterでその行をインストールするコンポーネント（オプション、必須ファイル、または条件付きセット。フォルダの場合は最初のファイルのコンポーネント）をメインウィンドウで選択します。
プレビューは、MODマネージャーと同じように、最初のステップの前にMOD情報ページでも開きます。ヘッダー画像（showImageとheightオプションに従う）、色付きのタイトル、作者、バージョン、ウェブサイトのリンク、そしてinfo.xmlの説明です。次へで最初の表示ステップへ進み、戻るでそこへ戻り、進捗バーはこれを最初のページとして数えます。
シナリオ行では、現在の選択とファイルの前提を名前を付けて保存し（fomod/scenarios/<名前>.jsonに、並べ替えに耐えるようオプション名で保存）、再読み込みや削除ができます。選択がどのオプションにも一致しなくなったシナリオは、見つからなかった数を報告します。シナリオフォルダが配布アーカイブに含まれることは決してありません。
検証は、どのオプションも設定しないフラグ値をテストしているために決して到達できないステップ、オプション、条件付きセットも、情報として報告するようになりました（ステップの場合：先行するステップにオプションがない）。
コマンドラインでは：ximod-architect simulate <root> [--scenario <name|file.json>] [--json]は、表示されるステップ、選択、サイズと上書きを含む最終インストール一覧、そして合計を表示します。batchのマニフェストエントリは"scenarios": [...]を持って再生できます（シナリオが一致しなくなるとエントリは失敗します）。

8.20 オプションごとのインストールサイズ

検証（バックグラウンドでファイルを計測します）の後、各オプションは、そのファイル表の上に、インストールサイズとファイル数を表示します（欠落しているソースは警告色で数えられます）。また、プロジェクトツリーにはオプション名の後にサイズが表示されます。サイズはファイルの更新時刻でキャッシュされ、検証のたびに更新されます。ximod-architect validate <root> --sizesは、オプション、必須ファイル、条件付きセットの合計を表示します。

8.21 Nexus説明文ジェネレーター

ツール →
Nexus用説明文…は、プロジェクトからMODのNexusページのテキストを構築します。タイトルと説明、要件（ゲームとMODが提供しない、インストールされたプラグインのマスター、および宣言されたファイル依存関係）、インストールオプション（ステップごとに1項目、選択タイプ付きのグループ、説明の最初の一文付きのオプション）、インストールに関する注意、そして前のバージョン…で選んだFOMODの以前のバージョンに照らして計算された変更履歴（追加・削除されたステップ・オプション・ファイル、メタデータの変更）です。BBCode（Nexus）またはMarkdownを選び、必要なセクションにチェックを入れ、プロジェクト内で見つかった翻訳サイドカーの中から言語を選んで、コピーまたは名前を付けて保存…します。コマンドラインでは：ximod-architect nexus-desc <root> [--format bbcode|md] [--previous <root>] [--lang <code>] [-o FILE]。

8.22 編集可能な条件エディター

§
8.7の条件エディターは、プロジェクトに作用するようになりました。すべての「設定元」と「使用元」の行は、メインウィンドウでノードを選択するリンクです。各フラグには、名前変更…（インラインフィールド。新しい名前は、そのフラグを設定するすべてのオプションと、それをテストするすべてのステップ表示・オプションパターン・条件付きセット・MOD要件に、取り消し可能な単一の変更として適用されます。別のフラグがすでに使っている名前は拒否されます）と、すべての使用を削除があります。各フラグの下には、設定される値とテストされる値が一覧表示されます。テストされるが一度も設定されない値は警告色で、設定されるが一度もテストされない値は灰色で表示されます。検証も同じ検出結果を報告します。どのオプションも設定しないテストされた値には警告（それをテストするノードを対象とする）、設定されるが一度もテストされないフラグには情報です。
ウィンドウ下部の条件ビルダーは、メインウィンドウで選択されたノードの条件 — ステップの表示可否、オプションの依存関係パターン（パターンを選ぶ、作成する、タイプを変える）、条件付きファイルセット、またはMOD要件 — を編集します。条件を文（IF flag A = B AND file C is Active THEN …）として、1条件につき1行の上に表示します。種類（フラグまたはファイル）、名前、値（プロジェクトから取得した候補付き。既知のフラグとその値、ファイル名、Active/Inactive/Missing）、削除ボタン、そして条件を追加です。変更はリアルタイムに適用され、取り消し可能です。
このエディターは独立したウィンドウになりました。メインウィンドウの横や別のモニターに移動でき、セッションをまたいで位置とサイズを保持し、タイトルバーのボタンで閉じます。フラグの一覧はプロジェクトツリーのように動作します。フラグの見出しを右クリックすると、展開（すべて／選択項目）と折りたたみ（すべて／選択項目）のサブメニューが開きます。テキストフィールドにフォーカスがない間は、↑と↓でカーソルリングを行から行へ（フラグの見出しとその「設定元」／「使用元」の行）移動し、→で閉じたフラグを展開（または開いたものの中に入り）、←で開いたフラグを折りたたみ（または行からそのフラグへ戻り）、Enterでその行をインスペクターに表示します（行のノード、またはフラグの最初の箇所）。プロジェクトツリーでも、Enterは同じことを行います。カーソルの下の行を選択し、インスペクターが追従します。

8.23 プロジェクトの文字列の表

ツール →
プロジェクトの文字列…は、プロジェクトのすべての名前と説明（MOD情報、ステップ、グループ、オプション）を1つの表に一覧表示します。場所、フィールド、テキストで、テキストはその場で編集できます（名前は1行、説明は複数行）。検索フィールド、種類フィルター（

すべて、名前、説明）、重複のみの切り替えで行を絞り込めます。同一のテキスト（大文字小文字と空白を無視）には×Nのバッジが付き、ホバーすると他の場所が表示されるので、コピー&ペーストされた説明を見つけるのに役立ちます。検索／置換の行（大文字小文字の一致、単語全体）で、現在の行または表示中のすべての行で置換します。場所をダブルクリックすると、メインウィンドウでノードが選択されます。コマンドラインでは：ximod-architect strings <root> [--duplicates] [--csv FILE]。

8.24 BSAおよびBA2アーカイブの読み取り

XIMODは、ベセスダのアーカイブを展開せずに、その目次を読み取れるようになりました。BSAのバージョン103（Oblivion）、104（Skyrim、Fallout 3/New Vegas）、105（Skyrim Special Edition）、そしてFallout 4とStarfieldのBA2一般（GNRL）アーカイブとテクスチャ（DX10）アーカイブ（バージョン1、2、3、7、8）です。Morrowindのアーカイブは分かりやすいメッセージとともに拒否されます。読み取られるのはエントリの名前とサイズのみで、圧縮されたエントリはその圧縮後のサイズとともに一覧表示されます。
これは3か所に反映されます。宛先衝突の検出（§ 8.1）は、オプションがインストールするアーカイブの中も見るようになりました。アーカイブに同じアセットを含む2つのオプションは警告として報告され（「ゲームのアーカイブのロード順が決めます」）、パックされていると同時にルースでインストールされるアセットは情報として報告されます（「ルースファイルが勝ちます」）。シミュレーターの最終ファイルツリー（§ 8.19）は、インストールされた各アーカイブの内容をその下に「(アーカイブ内)」と記して一覧表示し、合計はアーカイブにどれだけパックされているかを示します（二重には数えません）。そしてツール → アーカイブの内容…（またはファイル表のアーカイブの横にある目のボタン）は、エントリの表 — 形式、数、展開後の合計サイズ、パス、サイズ、圧縮後 — を検索フィールド付きで開きます。コマンドラインでは：ximod-architect archive <file.bsa|file.ba2> [--json]。validateも確実な宛先衝突を表示するようになりました。
実験的なBA2ライター（CLIのba2）は、その出力を新しいリーダーで読み戻すことで検証されるようになりました。パス、サイズ、名前／フォルダのハッシュが一致します。

8.25 入れ子の条件グループとバージョン条件

条件は、単一の演算子を持つ1つのフラットなリストに限られなくなりました。条件を持つすべての場所 — ステップの表示可否、オプションの依存関係パターン、条件付きファイルセット、MOD要件 — は、FOMODスキーマが許すとおり、条件と入れ子のグループ（それぞれ独自のAnd／Or演算子を持つ）を、任意の深さまで含められるグループになりました。「(DataVersion = Standard OR DataVersion = Undelayed) AND DataPreset = Aurelia」は、書かれたとおりに保持、読み込み、編集、保存されます（AureliaのUIとその234の条件付きセットで検証済み）。条件の種類も2つ追加されました。ゲームバージョン（gameDependency）とMODマネージャーバージョン（fommDependency）で、それぞれ最小バージョンを持ちます。
依存関係エディターでは、グループを追加が、独自の演算子、独自の条件、条件を追加／グループを追加／グループを削除ボタンを持つ入れ子グループを、字下げされた枠として作成します。条件ビルダー（§ 8.22）は、文の中で入れ子グループをかっこで囲んで表示し、同じコントロールで編集します。シミュレーター、検証（設定されないがテストされるフラグ、到達不能なステップ）、衝突検出、Nexus説明文は、いずれも入れ子グループを理解します。フラットなレイアウトで保存された古いプロジェクトテンプレートも引き続き読み込めます。

同じリリースでの2つの小さな変更：メニューバー上をポインターを滑らせて開いたメニューは、ポインターがそこを離れると閉じるようになりました（クリックで開いたメニューは、何かがクリックされるまで開いたままです）。また、プロジェクト自体の文字 ― 名前や説明の日本語、韓国語、中国語、アラビア語、タイ語など ― と、編集中の翻訳の文字を表示するのに必要なフォントが、インターフェースの言語にかかわらず自動的に読み込まれるようになり、翻訳ウィンドウやNexusウィンドウの言語一覧も含め、これらの文字が四角で表示されることがなくなりました。
メニューはファイル、編集、プロジェクト、ツール、ヘルプに整理され、日常的な操作用のツールバーがあり、F1で本マニュアルが開きます。グループの選択タイプ、オプションのタイプ、演算子、テーマは、平易な言葉のラベル（「正確に1つ（必須）」、「推奨」、「すべての条件（AND）」など）とホバー時の説明付きで表示され、オプションパネルの各セクションには、概念を説明するⓘマーカーがあります。確認ダイアログはウィンドウを暗くし、EnterとEscを受け付け、問題の一覧が長い場合はスクロールします。

8.26 ゲームとそのカテゴリの編集

ツール → ゲーム / カテゴリ… は、Categories.json
を手作業で編集することなく、Mod情報タブに表示されるゲームの一覧を管理するための独立したウィンドウを開きます。左側のペインにはゲームが一覧表示され、ゲームを追加、ゲームを削除、および 上へ移動 / 下へ移動
ボタンで一覧とその順序を変更できます。選択したゲームについて、エディターはその識別子（固有の内部キー）、名前、Nexus の slug（リンクおよび Nexus
の説明に使用）、ライトプラグインのレコード上限、基本マスターファイル（1 行につき 1 つ — そのゲーム自身の masters と公式 DLC）、および Nexus カテゴリ（1 行につき 1
つ、表示順）を示します。入力内容は入力中に検証され、識別子のないゲーム、重複した識別子、名前の欠落があると警告が表示され、すべての問題が解決されるまで 保存
は無効のままです。保存 はファイルを書き込み——書き込み可能であれば同梱の
assets/data/Categories.json
に、そうでなければアプリケーションが優先して読み込むユーザーごとのコピーに書き込みます——新しい一覧は直ちに反映されます。閉じる は確認後に未保存の編集を破棄します。

8.27 ツリーの色

ツール → ツリーの色…では、プロジェクトツリーの行の種類 —
MOD情報、ステップ、グループ、オプション、必須ファイル、条件付きインストール —
ごとに独自の色を割り当てられます。大規模なインストーラーの構造がひと目でわかるようになり、視力が衰えてきたときにも役立ちます（インターフェースのフォントサイズと組み合わせてください。§
4.5）。ウィンドウには6つの種類が、色見本、その色のアイコン、RGB値とともに一覧表示されます。「色をリセット」でデフォルトに戻ります。色のセットはテーマごとに1つあります。ウィンドウは現在使用中のテーマのセットを編集してその旨を表示し、ダークとライトを切り替えると（§
4.5）セットも切り替わります。色はConfig.ini（付録A）に保存されます。選択中の行は、選択背景の上でも読みやすいよう、常に標準のテキスト色のままです。
色見本をクリックすると「色の選択」ウィンドウが開きます。これはXIMOD
Translatorと同じカラーピッカーです。「インデックスカラー」タブにはAutoCADのカラーインデックスが表示されます。色相と濃淡からなる24 ×
10のグリッド、9つの標準色、グレーの階調、そして1から255までのインデックスを入力する「色：」フィールドです。「トゥルーカラー」タブには、色相／彩度エリア、輝度スライダー、H、S、Lの各フィールド、現在の色の赤・緑・青の値、そして255,128,0のような値を入力できる「RGB カラー：」フィールドがあります。「OK」で色がすぐに適用され —
ツリーはウィンドウの背後で再描画されます —
保存されます。「キャンセル」またはEscでは以前の色が保持されます。

8.28 ファイルダイアログが前回の場所を記憶

オプションにソースを追加するとき（「ファイルの追加」、「フォルダの追加」）や画像を参照するとき、ダイアログは毎回プロジェクトのルートから始まるのではなく、最後に選んだファイルまたはフォルダの親フォルダで開くようになりました。あるオプションにFeature
D\Aを選ぶと、次のダイアログはFeature
Dで開き、BとCがすぐそこにあります。フォルダ内のファイルを選ぶと、次のダイアログはそのフォルダで開きます。この記憶はセッションの間保持され、プロジェクトが変わるとリセットされます。記憶されたフォルダがルートの外にある場合や存在しなくなった場合、ダイアログはルートに戻ります。大規模なインストーラーでは、やはりエクスプローラーが最速です。機能フォルダを一度開き、各サブフォルダを対応するオプションにドロップしてください（§
8.5）。

8.29 プリセットウィザード

プロジェクト → プリセットウィザード…は、§
7.5で説明した「プリセット＋カスタマイズ」パターンを現在のプロジェクトに構築します。ユーザーはプリセットを選んでそれで終わりにすることも、プリセットを選んだうえでいくつかの機能を調整することも、すべてを手動で組み立てることもでき、しかもファイルが二重にインストールされることは決してありません。これを手作業で行うには、すべてのオプションへのフラグ、動的なタイプ、表示条件、そしてオプションごとに1つの条件付きインストールが必要です。ウィザードはそのすべてを、元に戻せる1回の操作で行います。
「既存のページから」タブでは、機能ページにあたるステップをチェックし（そのオプションがプリセットの選択対象になります）、生成される名前 ─
インストールモードのステップ、プリセットのグループ、カスタマイズのグループ、カスタマイズオプションのラベル、プリセットのフラグ、「なし」オプションの名前 ─
を確認してから、プリセットを定義します。「プリセットを追加」で列が作成され、マトリックスにはチェックしたすべてのページのすべてのオプションが行として表示されます。排他的なグループ（SelectExactlyOne／SelectAtMostOne）では、プリセットごとにオプションを1つ、または「（なし）」を選びます。複数選択のグループでは、いくつでもチェックできます。下部のバーには最初の問題（チェックされたページがない、プリセットがない、名前が欠けている、同じ名前のプリセットが2つある）が表示され、それが解決されるまで「適用」は無効のままです。
「適用」を押すと、まずインストールモードのステップが挿入されます（各プリセットはその選択のフラグを設定し、ページごとに1つの「カスタマイズ」オプションがそのページを開いてフラグをリセットします）。さらに、各機能ページは対応する「カスタマイズ」オプションがチェックされているときだけ表示されるようになり、オプションにフラグが追加され、プリセットが選んだバリアントはそのプリセットのもとでRecommendedとしてマークされます。排他的なグループには「なし」オプションが追加され（デフォルトでは推奨、何かを選ぶプリセットのもとでは任意）、オプションのファイルは「条件付きインストール」タブへ、フラグ値ごとに1つのセットとして移されます。メッセージに実行内容の概要が表示され、編集 → 元に戻すで操作全体を取り消せます。その後、プロジェクト → インストーラのプレビュー…（プリセットのみ、プリセット＋カスタマイズしたページ、ページのみ）と、このパターンを認識するプロジェクト → FOMODを検証（§
8.1）で結果を確認してください。
「新しいスケルトン」タブは、まだページのないプロジェクト向けです。各機能をページ名、選択の種類（排他的または複数）、1行に1つのバリアントで記述すると、「ページを作成」がそれらを ─ ファイルなしで ─
プロジェクトに追加し、それらのページにチェックを入れた状態で最初のタブに切り替わります。そこでプリセットを定義して適用し、その後でファイルを割り当てます。手動で選ぶものはページに、プリセットがインストールするものはウィザードが作成した条件付きインストールに割り当てます。

8.30 FOMOD間での切り取り、コピー、貼り付け

ステップ、グループ、オプションは、切り取り、コピー、貼り付けができます。編集 → 切り取り、コピー、貼り付け、ツリーのノードの右クリックメニューにある同じコマンド、またはテキストフィールドにフォーカスがないときのCtrl+X、Ctrl+C、Ctrl+Vを使います。テキストフィールド内では、これらのキーは引き続きテキストに作用します。コピーはノードをその内容すべてとともに取り込みます。つまり、グループとオプション、ファイル、画像、フラグ、条件、依存関係パターンです。切り取りはノードをプロジェクトから取り除いてクリップボードに保持します。編集 → 元に戻すで元の位置に戻せます。
貼り付けは選択に従います。ステップは選択したステップの後に（何も選択されていなければ末尾に）挿入され、グループは選択したステップの中の、選択したグループがあればその後に

、オプションは選択したグループの中の、選択したオプションの後に入ります。貼り付けたノードが選択状態になり、貼り付けは元に戻せる1つの変更として扱われます。選択項目がそのノードを受け入れられない場合（ステップが選択されていないときのグループ、グループが選択されていないときのオプション）は、何を選択すればよいかをメッセージが知らせます。
クリップボードはプロジェクトではなくアプリケーションに属しています。あるFOMODタブでコピーし、別のタブに切り替えて貼り付けてください。コピーはシステムのクリップボードにも置かれるため、あるXIMODウィンドウでコピーしたノードを別のウィンドウに貼り付けることもできます。パスはMODフォルダからの相対パスなので、別のMODから持ってきたノードは、このMODにないファイルを指していることがあります。貼り付け後、XIMODはそのノードのファイルのうちいくつがこのMODフォルダにないかを報告し（その後は、プロジェクト → 欠落ファイルを検索…、§ 8.32で参照先を付け替えられます）、ノードが判定に使うフラグのうち、このプロジェクトのどのオプションも設定しないものを一覧表示します。決して表示されないページや、タイプが決して変わらないオプションを、その場で見つけられます。

8.31 ディスク上でのソースの名前変更

ファイルエクスプローラーでファイルやフォルダの名前を変更すると、プロジェクトがそれに対して持つ参照がすべて壊れます。ファイル表（オプションのソース表（§ 4.2）、「必須のインストール」タブ（§ 4.3）、条件付きセットのファイル（§ 4.4））にある「ディスク上で名前を変更…」ボタンを使えば、代わりにXIMODから、選択したファイルまたはフォルダの名前を変更し、同時に参照も更新できます。更新されるのは、それを使用するすべてのオプション、必須ファイル、条件付きセット、そしてフォルダの場合はその中にあるものへのすべての参照です。このボタンは、プロジェクトにルートディレクトリが設定されると使用できるようになります。
ウィンドウには、現在のパス、新しい名前の入力欄（項目は同じフォルダに残ります）、そして追従する参照の数が表示されます。「名前を変更」またはEnterで適用します。Windowsで使用できない文字（: * ? や | など）を含む名前、ピリオドや空白で終わる名前、そのフォルダですでに使われている名前、もう存在しないソースは、ウィンドウ内で報告され、ウィンドウは開いたままになります。大文字・小文字だけの変更（main.esp → Main.esp）はどのシステムでも機能し、各参照は区切り文字のスタイル（\ または /）を保ちます。項目の名前はディスク上で即座に変更されます。編集 → 元に戻すで復元されるのはプロジェクトの参照であり、ディスク上の名前ではありません。

8.32 欠落ファイルの検索

ファイルはXIMODの外でも名前を変更されたり移動されたりします。MODを再編成しているときや、その新しいバージョンを展開した後などです。プロジェクト → 欠落ファイルを検索…はMODフォルダをスキャンし、プロジェクトが使用しているファイル、フォルダ、画像のうちもう存在しないものを、パスごとに1回、それを使用している参照の数とともに一覧表示します。
それぞれについて、ウィンドウはMODフォルダ内で見つかった候補を提示します。別のフォルダにある同じ名前（移動済み）、または元のフォルダにある、プロジェクトがまだ使用していない類似の名前（ファイルの場合は拡張子が同じもの。名前変更済み）です。同じ名前の項目がほかの場所にちょうど1つだけある場合は、それがあらかじめ選択されています。リストから別の候補を選ぶか、「（そのまま）」を選んで現状を維持するか、「参照…」をクリックしてMODフォルダ内の任意のファイルやフォルダを指定します。
「適用」は、選択したすべての参照を一度に（フォルダの場合はその中の参照も）、元に戻せる1つの変更として更新し、その後MODフォルダを再スキャンします。「再スキャン」は、自分でファイルを移動した後に同じ処理を行います。ファイル監視とは異なり、XIMODを閉じて

いる間に変わったものも検出でき、確認なしにプロジェクトを変更することは決してありません。それでも欠落したままのファイルは、プロジェクト → FOMODを検証（§ 4.8）が引き続き報告します。

付録

付録A — Config.iniファイル

設定は、実行ファイルの隣にあるConfig.iniファイルに保存されます。完全な例を次に示します。

```
; XIMOD Architect Configuration
[General]
Locale=fra
Country=FRA
FirstStart=1
Theme=Dark
FontSize=14
ReplaceNewlines=1
MaxRecentFiles=10
SplashScreenSeconds=2

[Window]
WindowWidth=1280
WindowHeight=800

[WindowPositions]
WinPos_ximod_translation=120,80
WinSize_ximod_translation=1100,700
WinPos_ximod_preview=200,140
WinSize_ximod_preview=900,650

[Scripts]
PreSaveScript=
PostSaveScript=

[RecentFiles]
RecentFile0=C:\Mods\MyMod
```

以前のキーに2つのキーが追加されました。Country（選択した国のISO 3166-1 alpha-3コード）とFirstStart（§ 2.5を参照）です。

[WindowPositions]セクションは、フリーウィンドウのジオメトリを記憶します（§ 2.6を参照）。各ウィンドウについて、WinPos_<id>がその位置（x,y）を、WinSize_<id>がそのサイズ（幅,高さ）を保存します。これらのエントリは、ユーザーが実際にウィンドウを移動またはサイズ変更して初めて作成されます。それがなければ、ウィンドウはメインウィンドウの中央に開きます。メインウィンドウのサイズは[Window]（WindowWidth／WindowHeight）に保持されますが、その位置は保持されません。

付録B — キーボードショートカット

macOSでは、CtrlキーはCmdに相当します。

グローバルショートカット

ショートカット	操作
Ctrl+N	新規プロジェクト。

ショートカット	操作
Ctrl+O	フォルダを開く。
Ctrl+Shift+O	ファイルを開く。
Ctrl+S	保存（FOMODを生成）。
Ctrl+,	設定を開く。
Ctrl+Q	アプリケーションを終了する。
F1	ユーザーマニュアルを開く（ヘルプ → ユーザーマニュアル）。
Ctrl+X / Ctrl+C / Ctrl+V	ツリーで選択したステップ、グループ、オプションを切り取り、コピー、または貼り付けします（§ 8.30）。テキストフィールドでは、これらのキーはテキストに作用します。

ウィンドウとダイアログ

ショートカット	操作
Esc	アクティブなフリーウィンドウ（国の選択、プロパティ、XMLエディター）またはモーダルダイアログ（設定、バージョン情報、確認、検証レポート、スクリプト）を閉じます。確認では「キャンセル」として働きます。翻訳エディターとメインウィンドウには効果がありません。

リスト操作 — プロパティと翻訳エディター

キー	操作
↑ / ↓	選択を1行上／下へ移動します。
Page Up / Page Down	1画面スクロールします。刻み幅はウィンドウの高さに合わせて調整されます。
Home / End	最初／最後の行へジャンプします（最後の行は完全に表示されるようになります）。
Enter	プロパティ：ハイライトされた項目を選択します。翻訳エディター：その行の入力フィールドにカーソルを置きます。
Tab / Shift+Tab	プロパティ：次／前の行（旗には移りません）。翻訳エディター（編集中）：次／前の翻訳フィールド。

旗グリッド — 「国の選択」ウィンドウ

キー	操作
↑ ↓ ← → (または 8 2 4 6)	グリッド内で選択枠を移動します。
Page Up / Page Down	旗を1画面スクロールします。
Home / End	最初／最後の旗へジャンプします（最後の行は完全に表示されるようになります）。
Tab / Shift+Tab	次／前の旗。
Enter	ハイライトされた国を確定します。

キー	操作
Esc	ウィンドウを閉じます。

付録C — 用語集

用語	定義
FOMOD	info.xmlとModuleConfig.xmlに基づくMODインストーラー形式。ベセスダのゲームの標準であり、他のほとんどのゲームでもVortexとMod Organizer 2によってサポートされています。
ステップ	1つ以上のグループを含む、インストールウィザードのページ。
グループ	選択タイプを共有するオプション（plugin）の集まり。
Plugin	ユーザーに提供されるオプション。インストールするファイルに関連付けられる。
フラグ	インストールを制御するために使われる条件フラグ（名前 = 値のペア）。
エンドニム	国または言語の、その言語で書かれた名称。
ISO 639-3	翻訳フォルダの名前となる3文字の言語コード（例：fra、eng）。
ISO 3166-1 alpha-3	3文字の国コード（例：FRA、USA）。
ルートディレクトリ	MODのファイルと、生成されるfomodサブフォルダを含む作業フォルダ。

付録D — 完全なFOMODの例

この付録では、架空だが完全なプロジェクト — Skyrim Special Edition向けの照明MOD「Dawnlight」 — を紹介し、XIMOD Architectの可能性の全範囲を実践します。必須ファイル、複数ステップ、ステップの表示条件、5つのグループタイプ、5つのオプションタイプ、条件フラグ、動的なオプションタイプ（依存関係パターン）、条件付きインストールです。以下の2つのファイルは1つのプロジェクトを構成し、XIMODが生成するとおりに再現しています（インデントは簡略化しています）。

D.1 info.xml

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>Dawnlight</Name>
    <Author>Patrice</Author>
    <Version>1.2.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/98765</Website>
    <Description>Improved lighting, candles and weather for Skyrim SE.</Description>
    <Groups>Visuals and Graphics</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

D.2 ModuleConfig.xml

この例でカバーされる機能：

- 必須ファイル：requiredInstallFiles（ファイル1つとフォルダ1つ）。
- ステップ1：SelectAllグループ（Requiredオプション）とSelectExactlyOne（Recommended／Optionalオプション、resフラグ）。
- ステップ2：SelectAnyグループ（candlesとmoonフラグ）とSelectAtMostOne。
- ステップ3：表示条件（<visible>）に続くSelectAtLeastOneグループ。動的タイプのオプション（dependencyType：candles=OnならRecommended、ファイルが欠落していればNotUsable）とCouldBeUsableオプションを含む。
- 条件付きインストール：2つのパターン（AndとOr演算子、フラグとファイルの依存関係、Active／Inactive／Missing状態）。

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Dawnlight</moduleName>
  <moduleImage path="fomod\header.png"/>
  <requiredInstallFiles>
    <file source="Dawnlight.esp" destination="Dawnlight.esp" priority="0"/>
    <folder source="core\meshes" destination="meshes" priority="0"/>
  </requiredInstallFiles>
  <installSteps order="Explicit">
    <installStep name="Main installation">
      <optionalFileGroups>
        <group name="Core module" type="SelectAll">
          <plugins order="Explicit">
            <plugin name="Base files">
              <description>Scripts and meshes essential to the mod.</description>
              <files>
                <folder source="core\scripts" destination="scripts" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Required"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
```

```
      <group name="Texture resolution" type="SelectExactlyOne">
        <plugins order="Explicit">
          <plugin name="2K">
            <description>2K textures, good quality/performance balance.</description>
            <image path="fomod\2k.png"/>
            <conditionFlags>
              <flag name="res">2K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\2k" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Recommended"/>
            </typeDescriptor>
          </plugin>
          <plugin name="4K">
            <description>4K textures, for powerful setups.</description>
            <image path="fomod\4k.png"/>
            <conditionFlags>
              <flag name="res">4K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\4k" destination="textures" priority="1"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="Ambience options">
    <optionalFileGroups>
      <group name="Lighting effects" type="SelectAny">
        <plugins order="Explicit">
          <plugin name="Dynamic candles">
            <description>Adds candles with flickering light.</description>
            <conditionFlags>
              <flag name="candles">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\candles" destination="meshes" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Enhanced moon">
            <description>Increases the intensity of the moonlight.</description>
            <conditionFlags>
              <flag name="moon">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\moon" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
      <group name="Sound ambience" type="SelectAtMostOne">
        <plugins order="Explicit">
          <plugin name="Night sounds">
            <description>Adds a nighttime sound ambience.</description>
            <files>
              <folder source="opt\sound" destination="sound" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
```

```
    <installStep name="ENB compatibility">
      <visible>
        <dependencies operator="And">
          <flagDependency flag="res" value="4K"/>
          <fileDependency file="KnownConflict.esp" state="Inactive"/>
        </dependencies>
      </visible>
      <optionalFileGroups>
        <group name="ENB preset" type="SelectAtLeastOne">
          <plugins order="Explicit">
            <plugin name="Light ENB">
              <description>Low-cost ENB preset.</description>
              <files>
                <file source="enb\light.ini" destination="enblocal.ini" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Optional"/>
              </typeDescriptor>
            </plugin>
            <plugin name="Ultra ENB">
              <description>High-end ENB preset (dynamic type).</description>
              <files>
                <file source="enb\ultra.ini" destination="enblocal.ini" priority="1"/>
              </files>
              <typeDescriptor>
                <dependencyType>
                  <defaultType name="Optional"/>
                  <patterns>
                    <pattern>
                      <dependencies operator="And">
                        <flagDependency flag="candles" value="On"/>
                      </dependencies>
                      <type name="Recommended"/>
                    </pattern>
                    <pattern>
                      <dependencies operator="Or">
                        <fileDependency file="EnbHelper.dll" state="Missing"/>
                      </dependencies>
                      <type name="NotUsable"/>
                    </pattern>
                  </patterns>
                </dependencyType>
              </typeDescriptor>
            </plugin>
            <plugin name="Experimental extension">
              <description>Additional effects, not guaranteed.</description>
              <files>
                <folder source="enb\extra" destination="textures" priority="2"/>
              </files>
              <typeDescriptor>
                <type name="CouldBeUsable"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
      </optionalFileGroups>
    </installStep>
  </installSteps>
  <conditionalFileInstalls>
    <patterns>
      <pattern>
        <dependencies operator="And">
          <flagDependency flag="res" value="4K"/>
          <flagDependency flag="candles" value="On"/>
        </dependencies>
        <files>
          <folder source="cond\candles4k" destination="textures" priority="0"/>
        </files>
      </pattern>
      <pattern>
        <dependencies operator="Or">
          <flagDependency flag="moon" value="On"/>
          <fileDependency file="SkyUI_SE.esp" state="Active"/>
        </dependencies>
        <files>
          <file source="cond\patch.esp" destination="patch.esp" priority="0"/>
        </files>
```

```
      </pattern>
    </patterns>
  </conditionalFileInstalls>
</config>
```

付録E — ライセンスとクレジット

XIMOD Architectのソースコードは、MITライセンス（© 2025-2026）のもとで配布されています。以下のサードパーティコンポーネントは、それぞれ独自のライセンスを保持します。完全な詳細と帰属は、アプリケーションに付属するTHIRD-PARTY-NOTICESファイルに記載されています。

コンポーネント	出典	ライセンス
XIMOD Architectのコード	—	MIT
Notoフォント	Google Fonts／Notoプロジェクト	SIL Open Font License 1.1
旗（SVG）	drapeauxdespays.fr	ロイヤリティフリー（商用利用を含む）
ISO 3166-1 alpha-3 国コード	ISO — iso.org/obp	自由にアクセス可能
ISO 639-3 言語コード	SIL — iso639-3.sil.org	帰属表示が必要（下記参照）
Rustライブラリ	crates.ioエコシステム	MIT／Apache-2.0（大部分）

ISO 639-3の帰属：言語コードはiso639-3.sil.orgに由来します（出典のクレジットが必要）。XIMODが追加したデータ — エンドニム、関連付けられたフォント、国 ↔ 言語の対応 — はISO 639-3標準の一部ではなく、標準の識別子は変更されていません。さらに、アプリケーションはコード一式を再配布する手段を一切提供してはなりません。